RAPPORT FONCTIONNEL SUR L'ECMMP

Ce document présente :

Une introduction et un rapport technique consacré au support NumWorks et à l'architecture de l'ECMMP (page 5 à 44).

Une veille technologique sur trois environnements de développement, suivie d'une conclusion générale (page 45 à 54).

Il est recommandé de lire l'introduction page 2 avant de lire le rapport de veille.

Création d'un éditeur de code modulaire en MicroPython

L'objectif de ce projet est de créer un environnement de développement modulaire en MicroPython, inspiré de VS Code et conçu pour fonctionner dans un environnement aux ressources limitées. Reproduire un tel système, même dans une version adaptée aux contraintes de la plateforme, représente un véritable défi technique.

Ce rapport présente le cheminement qui a conduit à la création d'un environnement de développement modulaire en MicroPython (ou du moins à ces détails actuels).

Ce projet a vu le jour en 2026. La calculatrice NumWorks, qui avait été délaissée pendant ma licence, a fait l'objet d'une redécouverte lors de mon arrivée en BTS, ce qui m'a amené à réaliser de nombreux programmes dans son environnement MicroPython : des jeux, des applications fonctionnelles, puis les premières ébauches du projet actuel.

Le concept d'environnement de développement modulaire en MicroPython, que nous désignerons désormais sous le nom d'« Environnement de Codage Modulaire en MicroPython », ou simplement « ECMMP », est un projet pour lequel je n'ai pas identifié de précédent au cours de mes recherches. Sa réalisation présente plusieurs difficultés. MicroPython est une version allégée de Python et la NumWorks fonctionne avec des ressources matérielles limitées, notamment en mémoire, en puissance de calcul et en stockage. Il faut donc concevoir l'ECMMP en tenant compte de ces contraintes, plutôt que de chercher à reproduire exactement un environnement de développement pour ordinateur. L'objectif est de proposer un espace de travail modulaire, capable d'accueillir différents langages et outils directement sur la calculatrice.

Pour autant, les caractéristiques matérielles de la NumWorks ne se résument pas à ses contraintes. Son microcontrôleur STM32, cadencé à 216 MHz, ses 256 Ko de SRAM et ses 8 Mo de mémoire Flash constituent des ressources sur lesquelles nous pouvons nous appuyer pour concevoir ce projet.

Ces caractéristiques ne suffisent toutefois pas, à elles seules, à déterminer les performances réelles d'une application comme notre IDE : celles-ci dépendent également du logiciel utilisé, de l'architecture du programme et des ressources effectivement disponibles lors de son exécution.

La NumWorks représente donc une plateforme intéressante pour expérimenter la conception d'un environnement de développement modulaire, tant sur le plan matériel que logiciel. Nous allons examiner les caractéristiques du support, la conception et le fonctionnement de l'ECMMP, puis les limites et les évolutions possibles du projet.

Table des matières

Introduction **2**
I. Étude du support **5**
A. Matériel : la calculatrice NumWorks **5**
1. Le microcontrôleur STM32 5
2. Limites physiques et contraintes de sécurité 6
B. Logiciel : l'environnement NumWorks **9**
1. Le système d'exploitation Epsilon 9
a. Une architecture logicielle adaptée aux ressources disponibles 9
b. Les fonctionnalités documentées dans la Toolbox 11
c. Les contraintes liées à l'exécution des programmes 12
2. L'environnement MicroPython 12
a. Les possibilités et les contraintes du module graphique Kandinsky 13
b. La compression des données graphiques 13
c. Les limites de MicroPython et l'optimisation des scripts 14
II. Architecture de l'ECMMP **16**
A. Cœur de l'IDE : première couche **16**
1. Position du cœur dans l'architecture générale 16
a. Un point central de coordination 16
b. Une séparation des responsabilités 17
c. Des responsabilités conservées par le cœur 18
2. Organisation des communications avec les modules 18
a. Un nombre limité de points d'entrée 18
b. La fonction d'initialisation 18
c. La fonction principale d'exécution 19
d. Une interface commune, des paramètres différents 19
3. Fonctionnement du cœur pendant l'exécution 20
a. Initialisation générale de l'IDE 20
b. La boucle principale d'exécution 20
c. Appels aux modules d'outils 21
d. Appels aux modules linguistiques 21
e. Organisation des traitements et actualisation de l'interface 22
4. Gestion du contenu de l'éditeur 22
a. Stockage centralisé du code source 22
b. Différencier le stockage et la représentation graphique 23
c. Organisation de l'affichage ligne par ligne 23
d. Intérêt de cette séparation 24
5. Les fonctions propres au cœur : conception de l'éditeur 24
a. Une interface qui ne dépend pas entièrement des modules 24
b. Séparation entre interface générale et contenu linguistique 25
c. Utilisation des fonctions graphiques de la NumWorks 25
6. Limites et perspectives d'évolution du cœur 25
a. Gestion de la mémoire 26
b. Coût de la transmission des données 26
c. Organisation de la boucle principale 27

d. Compatibilité et évolutivité des interfaces....27
e. Perspectives d'amélioration....27
B. Les modules de langages et d'outils : deuxième couche....28
1. Les modules de langages....29
a. Une architecture interne spécialisée....29
b. Analyse du contenu et identification des éléments du langage....30
c. Exploitation de la représentation intermédiaire....32
d. Gestion des résultats et communication avec l'interface....34
e. Gestion des interactions pendant l'exécution....35
2. Les modules d'outils....39
a. Organisation générale des modules d'outils....39
b. Le module de gestion du clavier : keyboard.py....40
c. Le module de gestion du curseur : mouse.py....42
d. Exploitation et coordination des modules d'outils....43
e. Limites et perspectives des modules d'outils....44
C. Compléments : troisième couche....45
1. Principe général des compléments....46
a. Une architecture ouverte à l'évolution....46
b. Des besoins qui n'ont pas nécessairement été anticipés....46
2. Compléments des modules d'outils....47
a. Modulation de modules pour des modules....47
b. Adaptation de l'architecture et préservation de sa cohérence....48
3. Compléments des modules linguistiques....49
a. Extension et création de modules originaux....49
b. Limites de conception et d'ajout de modules....49
4. Limites actuelles et perspectives....50
a. Une couche volontairement prospective....50
b. Le rôle de futurs développeurs....50
c. Une évolution conditionnée par les besoins réels....51
Rapport de veille technologique....53
1. Article 1 — ViperIDE : programmer depuis un navigateur....53
2. Article 2 — Pico VS Code : une extension pour simplifier le développement....55
3. Article 3 — Visual Studio Code sur Raspberry Pi....57
4. Synthèse générale....59
Conclusion générale....62
A. Bilan du rapport technique....62
B. Apports de la veille et perspectives....63

I. Étude du support

A. Matériel : la calculatrice NumWorks

1. Le microcontrôleur STM32

Présentation du microcontrôleur

Le fonctionnement de la NumWorks repose notamment sur un microcontrôleur STM32F730V8T6, équipé d'un cœur Arm Cortex-M7 pouvant fonctionner jusqu'à 216 MHz. Ce microcontrôleur intègre notamment 256 Ko de SRAM et 64 Ko de mémoire Flash interne. La calculatrice dispose également d'une mémoire Flash externe de 8 Mo, accessible au processeur par l'intermédiaire d'un contrôleur Quad-SPI. L'écran possède quant à lui son propre contrôleur, le ST7789V, auquel le microcontrôleur transmet les données d'affichage.

L'étude de cette architecture matérielle permet de distinguer plusieurs ressources dont les caractéristiques auront une incidence sur la conception de notre projet : la puissance de calcul, la mémoire vive disponible à l'exécution et les possibilités de stockage du programme et de ses données.

- **Puissance de calcul** : le STM32F730 dispose d'un cœur Arm Cortex-M7 cadencé jusqu'à 216 MHz et d'une unité matérielle de calcul en virgule flottante (FPU). Ces caractéristiques constituent une base intéressante pour exécuter des programmes MicroPython et gérer une interface graphique. Elles ne suffisent toutefois pas à garantir, à elles seules, la fluidité de l'IDE : celle-ci dépendra notamment de l'architecture logicielle, des opérations effectuées et de la fréquence de rafraîchissement de l'affichage (nous verrons cela plus tard).

- **Organisation de la mémoire vive** : le STM32F730 dispose de 256 Ko de SRAM, répartis entre 64 Ko de DTCM-RAM, 176 Ko de SRAM1 et 16 Ko de SRAM2. Il possède également 16 Ko d'ITCM-RAM, une mémoire destinée aux instructions critiques. La quantité de mémoire effectivement disponible pour un programme MicroPython ne peut cependant pas être déduite des seules capacités matérielles : elle dépend aussi de l'organisation d'Epsilon, de l'environnement d'exécution et des ressources mobilisées par le programme. Cette contrainte devra être prise en compte dans la conception de l'ECMMP, notamment pour limiter les allocations inutiles et éviter de charger des composants qui ne sont pas nécessaires.

- **Stockage externe via Quad-SPI :** la NumWorks possède une mémoire Flash externe de 8 Mo, reliée au microcontrôleur par une interface Quad-SPI pouvant fonctionner en mode *memory-mapped*. Ce mode permet au processeur d'accéder à la mémoire externe dans son espace d'adressage, notamment pour y lire des données et exécuter du code machine

compatible. Il ne transforme toutefois pas cette mémoire en SRAM et ne garantit pas que MicroPython puisse y charger ou compiler automatiquement des modules. L'utilisation effective de cette mémoire par l'ECMMP dépendra donc des possibilités offertes par le système logiciel et des mécanismes de stockage accessibles aux programmes.

2. Limites physiques et contraintes de sécurité

Si les spécifications théoriques du SoC STM32F730 définissent un cadre technique précis, l'expérimentation pratique et le développement de l'ECMMP mettent en lumière plusieurs contraintes physiques, thermiques et logicielles.

- Gestion de la mémoire et allocation dynamique sous MicroPython : Bien que le STM32F730 dispose de 256 Ko de SRAM, cette capacité physique ne correspond pas intégralement à la mémoire disponible pour les scripts MicroPython (cette dernière s'étend jusqu'à 43Ko). Lorsque j'ai expérimenté la possibilité d'afficher des images sur la NumWorks, via des formats d'encodage, notamment base 16, j'ai au début tenté de créer une unique chaîne de caractères contenant l'image complète et encodée en hexadécimal.

  La création d'une chaîne contenant l'ensemble de ces données, soit 38 404 caractères (4 octets spéciaux relatifs à la taille inclus) dans le format que j'ai conçu, a provoqué une saturation de la mémoire disponible pour le tas (heap) MicroPython et un plantage du système. Pour contourner cette difficulté, j'ai donc expérimenté une approche séquentielle, consistant à découper les données en chaînes de 320 caractères, regroupées dans une liste.

  Cette solution permet de limiter la taille de chaque chaîne individuelle, mais elle ne supprime pas du tout le coût mémoire associé au stockage de l'ensemble des données dans la liste (le découpage ajoute un coût lié aux objets et à la liste).

  Cette expérience montre l'importance de maîtriser les allocations dynamiques et la taille des objets manipulés. Elle met également en évidence l'intérêt potentiel d'une architecture modulaire comme l'ECMMP, dans laquelle les ressources nécessaires pourraient être chargées à la demande, tandis que celles devenues inutiles pourraient être libérées. Une telle approche permettrait de mieux maîtriser l'utilisation de la mémoire disponible, sans garantir à elle seule une réduction de la consommation mémoire.

- **Inertie thermique et dissipation en milieu confiné :** Contrairement aux appareils mobiles modernes intégrant des mécanismes avancés de gestion thermique, la NumWorks est un appareil compact dépourvu de ventilation active. Un environnement isolant peut limiter la dissipation de la chaleur produite lors d'une charge prolongée et entraîner une élévation de la température de surface.

  Lors d'un essai de dix minutes en charge, réalisé dans un environnement isolant, la température de surface de la NumWorks a augmenté continuellement, passant de 17.3°C

à 53.4°C. Celle d'un téléphone portable iPhone, en comparaison, est passée de 20.9° à 42.8°C, avant de redescendre et à 41.2°C en fin d'essai.

Des ralentissements ont été observés sur les deux appareils. Sur l'iPhone, la fréquence d'affichage a fortement diminué, jusqu'à environ 10 images par seconde, sans ralentissement apparent de la progression des calculs. Sur la NumWorks, le ralentissement affectait également la vitesse d'exécution des calculs et pas seulement la vitesse de rafraîchissement de l'écran. Les deux appareils ont donc présenté des comportements différents : sur le téléphone, la baisse de fluidité visuelle ne s'accompagnait pas d'un ralentissement apparent de la charge, tandis que sur la calculatrice, la progression du programme ralentissait également. Cette différence pourrait notamment s'expliquer par la manière dont chaque logiciel gère sa boucle d'exécution et son affichage, mais les observations réalisées ne permettent pas d'en identifier précisément la cause.

Les résultats doivent être interprétés avec prudence : les deux appareils exécutent des charges différentes et les températures mesurées en surface ne renseignent pas directement sur celle des composants internes. Cet essai exploratoire souligne néanmoins l'intérêt de concevoir l'ECMMP avec des boucles d'exécution efficaces, afin de limiter les traitements inutiles et la charge prolongée du processeur.

- **Gestion de l'énergie et contraintes liées à la batterie :** L'exécution de l'ECMMP directement sur la NumWorks mobilisera les ressources du microcontrôleur STM32F730. Le traitement du code, l'actualisation de l'interface et l'exécution des scripts pourront solliciter le processeur de manière prolongée, entraînant une consommation électrique accrue et une réduction potentielle de l'autonomie. Cette contrainte est particulièrement importante dans le cadre d'un environnement de développement, dont l'utilisateur peut solliciter les fonctions pendant de longues sessions.

  La gestion énergétique constitue donc un enjeu de conception pour l'ECMMP. Il sera nécessaire de limiter les traitements inutiles, d'éviter les boucles d'exécution actives lorsqu'aucune action n'est requise et de maîtriser la fréquence de rafraîchissement de l'interface. Ces optimisations permettront de réduire la charge de calcul superflue et pourront contribuer à limiter la consommation électrique ainsi que l'échauffement du système.

  Enfin, la possibilité d'utiliser l'ECMMP pendant la recharge soulève la question de la gestion thermique de l'appareil. La température observée dépend de plusieurs facteurs, notamment de la charge de calcul, de la consommation des autres composants et des conditions de recharge. En l'absence de mesures spécifiques concernant la batterie et le circuit de charge, il n'est toutefois pas possible d'affirmer que l'utilisation simultanée et la recharge présentent un risque particulier. L'ECMMP devra donc privilégier une exécution efficace et respecter les conditions d'utilisation préconisées par le fabricant.

B. Logiciel : l'environnement NumWorks

1. Le système d'exploitation Epsilon

Comme tout appareil électronique complexe, la NumWorks ne se résume pas à un boîtier contenant un écran, une batterie et un microcontrôleur. Elle constitue également un système informatique embarqué, doté d'un système d'exploitation conçu pour ses capacités matérielles et ses usages éducatifs : Epsilon.

Le code source d'Epsilon étant publiquement accessible, j'ai pu en étudier certains composants afin de mieux comprendre le fonctionnement logiciel de la calculatrice. Cette analyse permet d'identifier plusieurs caractéristiques qui exerceront une influence directe sur la conception de l'ECMMP.

a. Une architecture logicielle adaptée aux ressources disponibles

Epsilon repose sur une architecture logicielle organisée en plusieurs composants, notamment Ion, qui assure l'abstraction matérielle, Kandinsky, qui gère les opérations graphiques, et Escher, qui fournit des éléments d'interface utilisateur. L'ensemble contribue au fonctionnement des applications de la calculatrice.

L'étude du code source permet également d'examiner l'intégration de MicroPython et les options retenues pour configurer cet environnement. Contrairement à une installation complète de Python sur un ordinateur, l'environnement embarqué doit tenir compte des ressources limitées de la calculatrice. Certaines fonctionnalités peuvent donc être désactivées ou restreintes lors de la compilation afin d'adapter l'interpréteur aux contraintes de la plateforme.

L'examen des fichiers de configuration du port MicroPython permet notamment de repérer plusieurs fonctionnalités dont l'absence peut compliquer le développement de l'ECMMP.

- **La manipulation des données binaires (`struct` et `bytearray`) :** Le module `struct` permet notamment de convertir des données entre leur représentation binaire et des valeurs exploitables par un programme. Le type intégré `bytearray`, quant à lui, permet de manipuler des séquences d'octets modifiables. Lorsque ces outils sont indisponibles dans la configuration étudiée, certaines opérations de lecture, de conversion ou de traitement de données binaires deviennent plus difficiles à réaliser directement.

  Ces fonctionnalités auraient pu faciliter la manipulation de données graphiques ou de fichiers encodés. Leur absence impose donc d'étudier d'autres méthodes de représentation et de traitement des données, tout en tenant compte de la mémoire disponible.

- **L'organisation des données (*collections*, *array* et *attrtuple*)**

  Ces fonctionnalités répondent à des besoins différents : certaines facilitent l'organisation des données, tandis que d'autres permettent d'utiliser des structures adaptées à des types de valeurs particuliers ou d'accéder aux éléments d'un tuple par leur nom.

Lorsque ces outils ne sont pas disponibles, le développeur doit s'appuyer sur les structures restantes ou concevoir ses propres mécanismes. L'impact sur la consommation mémoire et les performances dépend toutefois de l'implémentation retenue : une structure de remplacement n'est pas nécessairement moins efficace dans tous les cas.

- **Accès aux informations d'exécution et à la gestion des flux (*sys*, io, *__file__*)**

  La disponibilité de ces fonctionnalités doit être examinée séparément. Le module *sys* fournit notamment des informations sur l'interpréteur et son environnement d'exécution, tandis que *io* regroupe des outils de gestion des entrées et sorties. L'attribut *__file__*, lorsqu'il est pris en charge, peut indiquer le chemin associé à un module.

  Des restrictions sur ces mécanismes peuvent compliquer la gestion des fichiers et la connaissance de l'environnement d'exécution. Pour l'ECMMP, cette question est particulièrement importante : un environnement de développement modulaire doit pouvoir organiser ses composants, retrouver ses ressources et charger les éléments nécessaires sans mobiliser inutilement la mémoire vive.

- **L'exécution asynchrone et la gestion de la mémoire (*async/await* et *gc*)**

  La prise en charge de la syntaxe *async/await* dépend notamment des options retenues lors de la compilation de MicroPython. Si cette fonctionnalité est désactivée dans la version étudiée, l'ECMMP ne pourra pas s'appuyer directement sur ce mécanisme natif pour organiser des tâches asynchrones.

  Cela ne signifie pas pour autant qu'une organisation coopérative des tâches soit impossible : d'autres mécanismes peuvent être envisagés, selon les fonctionnalités effectivement disponibles.

  Le ramasse-miettes, ou *garbage collector* (gc), doit également être pris en compte. Il permet de récupérer la mémoire occupée par des objets qui ne sont plus accessibles par le programme. Son efficacité dépend cependant de la manière dont les objets et les références sont gérés. Il ne peut pas libérer un objet qui reste référencé, même si le développeur n'en a plus besoin sur le plan fonctionnel.

  Ces observations mettent en évidence une difficulté centrale : la conception de l'ECMMP devra s'adapter aux fonctionnalités réellement disponibles dans l'environnement embarqué, plutôt que de supposer que l'ensemble des outils d'un Python classique est accessible.

  Il ne s'agit donc pas simplement de reproduire les fonctions d'un éditeur de code sur ordinateur. Il faudra concevoir une architecture logicielle capable de fonctionner dans un environnement plus contraint, en limitant les allocations inutiles, en organisant le chargement des ressources et en tenant compte des limites de l'interpréteur.

b. Les fonctionnalités documentées dans la Toolbox

L'environnement de programmation de la NumWorks intègre une Toolbox permettant d'accéder à des fonctions, des variables, des symboles, des structures de contrôle et aux fonctions de différents modules.

En parcourant cette arborescence, il est notamment possible de consulter les fonctions proposées par un module, les paramètres présentés dans l'aide intégrée et leur utilité. Par exemple, le chemin « Toolbox → Modules → Kandinsky » permet de découvrir plusieurs fonctions liées au dessin et à l'affichage graphique.

La fonction *draw_string(text, x, y)* permet ainsi d'afficher une chaîne de caractères à des coordonnées données. Toutefois, l'aide visible dans la Toolbox ne présente pas nécessairement l'ensemble des paramètres que la fonction peut accepter. Dans le cas de *draw_string*, des paramètres supplémentaires permettent notamment de préciser les couleurs utilisées pour le texte et son arrière-plan.

Cette différence entre la documentation intégrée et les possibilités effectives des fonctions constitue un élément important pour le développement de l'ECMMP. Un environnement de développement doit non seulement permettre d'écrire du code, mais également aider l'utilisateur à découvrir les fonctionnalités disponibles et à comprendre comment les utiliser.

L'étude de la Toolbox permet ainsi d'identifier les fonctions sur lesquelles l'ECMMP pourra s'appuyer, tout en mettant en évidence les limites de l'aide fournie à l'utilisateur. Mes expérimentations montrent également que plusieurs fonctions peuvent produire des résultats similaires avec des performances différentes. Ces différences devront être évaluées lorsqu'elles auront un impact sur la fluidité de l'interface ou sur la consommation des ressources.

c. Les contraintes liées à l'exécution des programmes

Un dernier élément important concerne la manière dont les scripts MicroPython sont exécutés au sein d'Epsilon.

Contrairement à un programme qui dispose de ressources dédiées à son exécution, un script lancé sur la NumWorks fonctionne dans un environnement partagé avec les autres responsabilités du système. La gestion des interactions avec le clavier, l'affichage et le fonctionnement général de la calculatrice doivent rester compatibles avec l'exécution du programme.

L'analyse des fichiers de configuration de MicroPython, notamment de la macro MICROPY_VM_HOOK_LOOP, permet d'étudier les mécanismes susceptibles de relier l'exécution de la machine virtuelle aux opérations du système. Le comportement exact de ce mécanisme dépend toutefois de son implémentation dans le port étudié : sa seule présence ne permet pas de conclure que chaque itération d'une boucle Python provoque nécessairement un ralentissement mesurable.

Sur le plan pratique, une boucle qui effectue continuellement des calculs ou actualise l'écran sans nécessité peut mobiliser inutilement les ressources disponibles. À l'inverse, une interface qui limite ses rafraîchissements aux changements utiles peut éviter une partie de ces traitements superflus.

Cette contrainte est particulièrement importante pour l'ECMMP. Un environnement de développement doit pouvoir gérer l'affichage de son interface, la navigation entre ses composants et les opérations liées au code sans rendre l'utilisation de la calculatrice excessivement lente.

Il sera donc nécessaire de concevoir des boucles de traitement efficaces, de limiter les rafraîchissements inutiles et de maîtriser les allocations de mémoire. Ces choix devront permettre de concilier les fonctions attendues d'un environnement de développement avec les capacités réelles de la NumWorks.

2. L'environnement MicroPython

L'environnement MicroPython constitue la couche logicielle sur laquelle repose le développement d'ECMMP. Bien qu'il conserve une grande partie de la syntaxe et des mécanismes du langage Python, MicroPython est conçu pour fonctionner sur des systèmes embarqués disposant de ressources limitées. Il ne propose donc pas nécessairement toutes les fonctionnalités de Python standard, et son utilisation sur la calculatrice NumWorks impose de tenir compte à la fois des capacités du langage, des modules disponibles et des contraintes matérielles du support.
Dans le cadre d'ECMMP, trois aspects de cet environnement sont particulièrement importants : la gestion des ressources graphiques, les possibilités de compression des données visuelles et l'optimisation des programmes afin de limiter leur occupation de l'espace de stockage.

a. Les possibilités et les contraintes du module graphique Kandinsky

La NumWorks fournit le module *kandinsky*, une bibliothèque graphique spécifique qui permet notamment d'afficher du texte, de colorer des pixels et de remplir des rectangles. Elle met à disposition des fonctions telles que *draw_string(), set_pixel()* et *fill_rect()*, ainsi qu'une fonction *color*() permettant de définir une couleur à partir de ses composantes rouge, verte et bleue. Ces fonctions constituent une base suffisante pour concevoir une interface graphique personnalisée sans devoir développer un système d'affichage de bas niveau.

Cependant, l'utilisation du graphisme doit rester mesurée dans le cadre d'ECMMP. L'écran de la calculatrice possède une résolution de 320 × 240 pixels, soit 71 040 pixels au total. Une modification graphique répétée, notamment lorsqu'elle implique de nombreux appels individuels à `set_pixel`, peut entraîner un coût d'exécution important. Il est donc préférable de privilégier des opérations groupées, comme le remplissage de rectangles, et de limiter le redessinage seulement aux zones qui doivent réellement être actualisées, de plus on peut noter qu'il est légèrement plus rapide d'appeler `fill_rect` pour dessiner un rectangle de 1 x 1 que d'appeler `set_pixel`.

Cette contrainte influence directement les choix d'interface du projet. ECMMP étant un environnement destiné à l'édition de code, il n'est pas nécessaire de lui ajouter des éléments visuels complexes ou des animations coûteuses. Une interface sobre, composée de zones de texte, de séparateurs et de quelques éléments graphiques simples, répond davantage aux objectifs du projet tout en limitant les opérations effectuées par le programme.

b. La compression des données graphiques

Une autre contrainte concerne la représentation des images dans les scripts. Une image stockée directement sous la forme d'une longue chaîne hexadécimale peut occuper un espace important, même lorsque cette méthode facilite sa manipulation en Python.

Une première expérimentation a consisté à représenter une image monochrome par une chaîne hexadécimale contenant également ses dimensions. Dans l'exemple étudié, les quatre premiers caractères, A078, correspondent à la largeur et à la hauteur de l'image : A0 représente 160 en hexadécimal et 78 représente 120. Les dimensions peuvent ainsi être récupérées directement depuis le début de la chaîne avant de procéder au décodage des données graphiques.

Cette expérimentation constitue une première preuve de faisabilité pour le stockage d'une image sous forme de texte. Elle a également permis d'identifier les limites d'une représentation qui consomme plusieurs caractères pour chaque pixel ou groupe de pixels.

Une évolution envisagée pour ECMMP consiste à appliquer un principe similaire à des images en couleur, en réduisant le nombre de couleurs utilisables à 4 096. Ce choix correspond à (2^{12}), soit 12 bits d'information par pixel. Un alphabet de 64 symboles permettrait théoriquement de représenter ces 12 bits à l'aide de deux symboles, puisque ($64^2 = 4096$).

Il ne s'agirait toutefois pas d'appliquer directement l'encodage Base64 standard à chaque pixel, mais de définir un encodage personnalisé reposant sur un alphabet de 64 caractères. Une fonction de décodage dédiée devrait ensuite reconstruire les valeurs associées aux pixels et les transmettre aux fonctions graphiques de Kandinsky. Cette solution demeure une proposition d'architecture : contrairement à l'encodage hexadécimal monochrome, son fonctionnement n'a pas encore été validé expérimentalement dans le cadre du projet.

Cette méthode pourrait réduire la longueur des chaînes représentant les images par rapport à un encodage hexadécimal plus classique. Son intérêt réel dépendrait néanmoins du format de comparaison choisi, du coût du décodage et de la mémoire nécessaire à l'affichage. L'objectif ne serait pas de multiplier les images dans ECMMP, mais de disposer d'une méthode compacte pour quelques éléments graphiques utiles à l'interface.

c. Les limites de MicroPython et l'optimisation des scripts

L'utilisation de MicroPython sur la NumWorks implique également de tenir compte des fonctionnalités réellement disponibles dans la version intégrée à Epsilon. Certaines bibliothèques ou fonctionnalités présentes dans une installation Python classique peuvent être absentes ou configurées différemment. Il est donc nécessaire de vérifier leur disponibilité sur la calculatrice plutôt que de supposer qu'un programme fonctionnant sur ordinateur pourra être exécuté sans adaptation.

Ces différences influencent la conception des modules d'ECMMP. Chaque fonctionnalité doit être développée en fonction des outils effectivement accessibles, en évitant les dépendances inutiles et les mécanismes trop coûteux pour l'environnement ciblé.

L'optimisation du code source constitue un autre levier. En Python, il est possible d'importer une fonction sous un alias afin d'utiliser un nom plus court dans le reste du programme. Par exemple :

```
from kandinsky import fill_rect as f
```

Les appels peuvent ensuite être écrits sous la forme f(...) plutôt que fill_rect(...). Le nom de la fonction passe ainsi de neuf caractères à un seul, soit une différence de huit caractères par occurrence dans le texte source. Pour trente appels, l'économie théorique atteint donc 240 caractères.

Si chaque caractère du fichier occupe un octet dans la représentation actuelle de la NumWorks, cette différence correspond à 240 octets. Rapportée à un espace de stockage de 43 Ko, elle représente une économie limitée, de l'ordre de 1/180 si l'on prend 43 000 octets comme référence. Cette estimation doit être interprétée comme une comparaison de longueur du code source : l'économie réelle dépend du mode de stockage des scripts et de leur représentation interne.

L'intérêt de cette technique devient plus perceptible lorsque plusieurs modules sont développés. ECMMP étant conçu pour accueillir des outils et, éventuellement, plusieurs modules linguistiques, les petites optimisations peuvent s'additionner à mesure que le projet grandit. Elles ne remplacent cependant pas les optimisations structurelles : réduire les dépendances, limiter les données inutiles et éviter de charger des modules qui ne sont pas nécessaires restent des moyens plus généraux de maîtriser les ressources.

De plus, certaines fonctions couramment associées à d'autres langages de programmation ne sont pas directement utilisables dans l'environnement Python de la NumWorks. C'est notamment le cas de printf(), dont l'utilisation a été testée sans succès dans l'environnement étudié. Cette différence impose d'adapter le code aux fonctions effectivement disponibles dans la version de MicroPython intégrée à la calculatrice

Enfin, il convient de distinguer l'espace occupé par les scripts de la mémoire vive utilisée pendant leur exécution. Le ramasse-miettes de MicroPython, lorsqu'il est disponible et actif, récupère la mémoire des objets qui ne sont plus accessibles ; il ne réduit pas directement la taille des fichiers stockés. La conception d'ECMMP doit donc considérer séparément la capacité de stockage des scripts, la mémoire vive nécessaire à leur chargement et leur exécution, ainsi que le coût des opérations graphiques.

En définitive, MicroPython offre suffisamment de possibilités pour développer une interface personnalisée sur la NumWorks, mais son utilisation exige des choix adaptés aux ressources disponibles. La sobriété graphique, l'encodage compact des données et la réduction des dépendances constituent ainsi des pistes complémentaires pour construire un environnement modulaire qui reste exploitable sur la calculatrice.

II. Architecture de l'ECMMP

A. Cœur de l'IDE : première couche

Le cœur constitue la première couche de l'architecture de l'ECMMP. Il représente le point central de l'environnement, autour duquel s'organisent les différents modules de langages et d'outils.

Son rôle principal est d'assurer la coordination générale du programme. Il prend en charge l'initialisation de l'environnement, la gestion de son fonctionnement et les échanges avec les composants spécialisés. Il est également responsable d'une partie du fonctionnement propre de l'éditeur, notamment de son organisation graphique et de la gestion de son contenu.

Le cœur peut ainsi être considéré comme le chef d'orchestre de l'ECMMP : il ne réalise pas nécessairement tous les traitements lui-même, mais il détermine quand et comment les différents composants interviennent. Il conserve néanmoins des responsabilités qui lui sont propres et ne dépend donc pas exclusivement des modules externes à sa première couche.

Cette architecture repose sur une volonté de séparation des responsabilités. Le cœur doit conserver une structure aussi générale que possible, tandis que les traitements spécifiques sont délégués aux composants adaptés. Cette distinction permet d'envisager l'évolution de l'IDE sans avoir à modifier systématiquement son fonctionnement central.

1. Position du cœur dans l'architecture générale

a. Un point central de coordination

Le cœur est la partie autour de laquelle s'organise l'ensemble de l'ECMMP. Il assure la liaison entre les différents composants et constitue leur principal point d'entrée depuis le programme central.

Cette position explique son classement dans la première couche : il ne s'agit pas d'un module spécialisé parmi d'autres, mais de la structure qui permet leur fonctionnement commun.

Les modules de langages et les modules d'outils interviennent donc sous son contrôle. Le cœur décide du moment où ils doivent être initialisés ou sollicités, leur transmet les informations nécessaires et récupère, lorsque cela est prévu, les données qu'ils retournent.

Cette organisation évite que les différents modules aient besoin de gérer eux-mêmes l'intégralité du fonctionnement de l'IDE. Ils peuvent se concentrer sur leurs responsabilités respectives, tandis que la coordination demeure centralisée.

Le fonctionnement général repose ainsi sur une relation entre un composant central et plusieurs composants spécialisés. Cette relation permet de conserver une architecture lisible, dans laquelle chaque élément possède un rôle identifiable, c'est pourquoi le cœur correspond à l'élément central de l'architecture et porte le qualificatif de « première couche ».

b. Une séparation des responsabilités

La conception du cœur repose sur une distinction entre deux catégories de traitements.

La première regroupe les opérations qui relèvent directement du fonctionnement général de l'éditeur. Elles sont prises en charge par le cœur lui-même.

La seconde regroupe les opérations spécialisées qui nécessitent l'intervention d'un module. Le cœur ne reproduit alors pas le traitement : il transmet les paramètres nécessaires au composant concerné et lui délègue son exécution.

Cette distinction permet notamment d'éviter de mélanger les mécanismes propres à un langage avec les fonctions générales de l'interface.

Le cœur n'a donc pas vocation à connaître les détails internes de chaque module. Il doit uniquement connaître les fonctions qu'il peut appeler, les paramètres qu'elles attendent et la nature des informations qu'elles peuvent retourner.

Cette approche limite les dépendances entre les différentes parties du programme. Un module peut ainsi être modifié dans son fonctionnement interne sans que cela entraîne nécessairement une modification du cœur, à condition que son interface de communication reste compatible.

c. Des responsabilités conservées par le cœur

La modularité de l'ECMMP ne signifie pas que l'intégralité de ses fonctionnalités doit être externalisée.

Le cœur conserve plusieurs opérations qui lui sont propres, notamment celles qui concernent l'organisation générale de l'éditeur et son apparence.

Il peut ainsi assurer directement la gestion de certains éléments graphiques, la construction de l'interface et l'organisation de ses différentes zones.

Cette répartition permet de distinguer l'interface générale de l'éditeur des traitements spécialisés réalisés par les modules linguistiques.

Le cœur reste donc un véritable composant fonctionnel. Il n'est pas uniquement un intermédiaire chargé de transmettre des informations aux autres modules.

2. Organisation des communications avec les modules

a. Un nombre limité de points d'entrée

Afin de simplifier les échanges entre le cœur et les modules, le fonctionnement retenu repose sur deux points d'entrée principaux au maximum pour chaque module.

Le premier correspond à la fonction d'initialisation. Elle permet de préparer l'environnement propre au module et de retourner les informations nécessaires à son utilisation.

Le second correspond à sa fonction principale d'exécution. Elle est sollicitée lorsque le cœur souhaite faire intervenir le module dans le fonctionnement de l'IDE.

Cette organisation permet de conserver une interface de communication simple et prévisible.

Du point de vue du cœur, il n'est donc pas nécessaire de connaître l'ensemble des fonctions internes d'un module. Celui-ci peut posséder ses propres fonctions auxiliaires pour organiser ses traitements, mais le cœur n'a besoin de connaître que ses points d'entrée principaux.

Cette limitation facilite également l'intégration de nouveaux composants. Tant que le module respecte les conventions de communication prévues, il peut être sollicité de la même manière que les autres.

b. La fonction d'initialisation

La fonction d'initialisation intervient lors de la préparation de l'environnement.

Elle permet au module de construire ou de récupérer les données dont il aura besoin pendant son fonctionnement. Ces informations peuvent ensuite être transmises au cœur sous la forme d'une structure de données adaptée.

Le cœur conserve les données retournées afin de pouvoir les réutiliser lors des appels suivants.

Cette méthode permet notamment de maintenir l'état d'un module entre plusieurs itérations de la boucle principale, sans devoir recommencer systématiquement son initialisation.

La fonction d'initialisation constitue ainsi une étape distincte de l'exécution courante du module. Elle prépare les conditions nécessaires à son utilisation ultérieure.

c. La fonction principale d'exécution

La seconde fonction constitue le point d'entrée utilisé lorsque le cœur souhaite solliciter le module.

Contrairement à l'initialisation, elle intervient pendant le fonctionnement de l'IDE. Elle reçoit les informations nécessaires au traitement demandé, effectue les opérations relevant du module et retourne les résultats utiles à la poursuite de l'exécution.

Les paramètres transmis dépendent de la nature du composant concerné.

Cette organisation permet au cœur de conserver le contrôle du déroulement général tout en laissant chaque module gérer ses opérations internes.

Le fonctionnement reste donc centralisé au niveau de l'orchestration, mais décentralisé au niveau de la réalisation des traitements spécialisés.

d. Une interface commune, des paramètres différents

Le nombre limité de points d'entrée ne signifie pas que tous les modules doivent recevoir les mêmes paramètres.

Un module d'outil et un module linguistique ne répondent pas aux mêmes besoins. Le cœur leur transmet donc des informations différentes, adaptées à leur fonction.

Le principe commun concerne uniquement la manière de communiquer avec eux : initialiser leur environnement, puis solliciter leur fonction principale lorsque cela est nécessaire.

Cette distinction permet de conserver une architecture uniforme sans imposer une structure de données identique à des composants dont les responsabilités sont différentes.

3. Fonctionnement du cœur pendant l'exécution

a. Initialisation générale de l'IDE

Au lancement de l'ECMMP, le cœur commence par préparer son propre environnement.

Cette étape comprend notamment l'initialisation des données nécessaires à l'éditeur et la préparation des éléments graphiques qui composent son interface.

Il procède ensuite à l'initialisation des modules nécessaires en sollicitant leur fonction dédiée.

Les données retournées sont conservées afin de maintenir les informations nécessaires à leur fonctionnement pendant toute la durée d'utilisation de l'IDE.

Une fois cette préparation effectuée, le programme peut entrer dans sa boucle principale.

Cette phase de démarrage permet de distinguer clairement la construction initiale de l'environnement de son fonctionnement courant.

b. La boucle principale d'exécution

Le fonctionnement de l'ECMMP repose sur une boucle principale qui assure la continuité de l'interface.

Elle permet au cœur de traiter successivement les interactions de l'utilisateur, d'actualiser les données de l'environnement et de solliciter les modules concernés.

À chaque itération, le cœur vérifie les informations nécessaires au fonctionnement des composants, puis leur transmet les paramètres adaptés.

Les opérations sont réalisées de manière successive, selon l'organisation définie par le programme principal.

La boucle constitue donc le mécanisme central de coordination de l'IDE. Elle permet de maintenir un fonctionnement continu sans confier à chaque module la responsabilité de gérer l'ensemble de l'application.

Elle offre également un point unique à partir duquel les différents traitements peuvent être organisés et optimisés.

c. Appels aux modules d'outils

Les modules d'outils sont principalement sollicités pendant l'exécution de la boucle principale.

Le cœur leur transmet les informations dont ils ont besoin pour réaliser leurs traitements. Ces paramètres peuvent notamment correspondre à des données relatives à l'état de l'interface ou aux interactions détectées.

Le module reçoit ainsi les informations qui concernent directement sa responsabilité, sans avoir à accéder à l'ensemble des données internes du cœur.

Cette transmission ciblée permet de limiter les dépendances entre les composants et de conserver une bonne séparation des traitements.

Le cœur reste responsable de l'organisation des appels : il détermine le moment où le module doit intervenir et exploite les informations retournées lorsque cela est nécessaire.

d. Appels aux modules linguistiques

Le fonctionnement des modules linguistiques présente une particularité : leur intervention concerne principalement le traitement et la représentation du contenu de l'éditeur.

Le cœur conserve le programme saisi par l'utilisateur dans une chaîne de caractères unique, qui constitue le stockage central du code source.

Lorsqu'un traitement linguistique ou un rendu du contenu est nécessaire, le cœur transmet cette chaîne à la fonction principale du module correspondant.

Le module linguistique peut alors effectuer ses propres opérations sur le contenu transmis, notamment les traitements nécessaires à sa représentation graphique.

Cette méthode permet de conserver le contenu du programme indépendamment de la manière dont il est affiché.

Le cœur n'a donc pas besoin d'intégrer directement les particularités de chaque langage. Il se contente de transmettre les données au module approprié, qui prend en charge le traitement correspondant.

Une évolution envisagée consiste à effectuer un découpage préalable du contenu en unités lexicales, appelées tokens. Cette étape permettrait de structurer les informations transmises au module et de lui éviter de devoir traiter systématiquement un bloc de texte brut particulièrement volumineux.

Le fonctionnement exact de cette préparation dépendra du format retenu et des besoins des différents modules linguistiques.

e. Organisation des traitements et actualisation de l'interface

Les différents composants n'interviennent pas nécessairement de la même manière ni au même moment.

Certains sont sollicités pendant le fonctionnement courant de la boucle principale, tandis que d'autres interviennent lorsqu'une opération particulière doit être effectuée, notamment lors du traitement du contenu de l'éditeur.

Cette distinction permet d'éviter de confondre la gestion permanente de l'interface avec les traitements ponctuels associés à certaines fonctionnalités.

Le cœur conserve ainsi la maîtrise de l'ordre d'exécution et détermine les opérations nécessaires à chaque étape.

Cette organisation doit également permettre d'éviter les traitements redondants. Par exemple, une opération de rendu ne devrait pas nécessairement être répétée lorsque les données qui la déterminent n'ont pas changé.

L'optimisation de ces appels constitue donc un aspect important de l'évolution du fonctionnement général.

4. Gestion du contenu de l'éditeur

a. Stockage centralisé du code source

Le contenu de l'éditeur est conservé dans une chaîne de caractères unique.

Cette chaîne constitue la représentation persistante du programme saisi par l'utilisateur en mémoire. Elle contient l'ensemble du texte, y compris les espaces et les caractères de contrôle nécessaires à l'organisation du code.

On peut la considérer comme le tampon textuel principal de l'éditeur, ou plus simplement comme son « giga string ».

Cette méthode présente l'avantage de disposer d'une seule source de référence pour le contenu du programme.

Les différentes opérations de représentation peuvent ainsi exploiter la même donnée sans avoir à maintenir plusieurs copies complètes du texte.

Le cœur conserve la responsabilité de ce stockage, tandis que les composants chargés du traitement linguistique peuvent recevoir le contenu lorsqu'ils doivent intervenir.

b. Différencier le stockage et la représentation graphique

Il est important de distinguer le stockage du code source de sa représentation à l'écran.

La chaîne principale conserve le contenu intégral du programme. Elle ne constitue toutefois pas directement la structure utilisée pour effectuer chaque opération graphique.

Pour afficher correctement le texte, celui-ci doit être organisé selon les besoins de la fonction de rendu.

Une fonction d'affichage peut ainsi découper temporairement la chaîne principale en plusieurs chaînes plus petites, notamment à partir des caractères de retour à la ligne.

Cette opération permet de disposer d'une liste de chaînes correspondant aux différentes lignes du programme.

Cette liste constitue une structure temporaire, propre à la fonction qui réalise le rendu. Elle n'a pas vocation à remplacer la chaîne principale ni à devenir un second stockage permanent du code.

Une fois l'opération d'affichage terminée, la variable locale et les références qu'elle contient ne sont plus nécessaires. Elles peuvent donc être libérées lorsque le ramasse-miettes de MicroPython récupère les objets qui ne sont plus accessibles.

Cette distinction permet de conserver un stockage centralisé tout en utilisant temporairement des structures adaptées aux besoins de l'affichage.

c. Organisation de l'affichage ligne par ligne

L'affichage d'un programme contenant plusieurs lignes nécessite de contrôler leur positionnement à l'écran.

Une représentation sous forme de chaîne unique ne suffit pas toujours à obtenir le résultat graphique attendu, notamment lorsque les coordonnées de chaque ligne doivent être maîtrisées.

Le découpage du contenu en chaînes distinctes permet alors de traiter les lignes individuellement.

La fonction de rendu peut parcourir la liste obtenue et transmettre successivement chaque chaîne à draw_string(), en adaptant les coordonnées d'affichage.

Cette méthode permet notamment de conserver un positionnement cohérent entre les différentes lignes et de mieux maîtriser la représentation du contenu dans l'espace disponible.

Elle s'inscrit dans la logique générale de séparation entre les données persistantes et les structures temporaires utilisées pour produire l'affichage.

d. Intérêt de cette séparation

Cette organisation présente plusieurs avantages.

Premièrement, le contenu principal reste indépendant de son apparence. Une modification de la présentation ne nécessite donc pas de modifier la structure utilisée pour conserver le code source.

Deuxièmement, les structures temporaires sont créées uniquement lorsqu'elles sont nécessaires. Elles ne sont pas conservées inutilement pendant toute la durée d'utilisation de l'IDE.

Enfin, cette séparation facilite l'intégration des modules linguistiques. Le cœur peut conserver le contenu dans un format général, tandis que chaque module reste libre d'organiser son traitement et son affichage selon ses propres besoins.

Le stockage, le traitement et la représentation graphique sont ainsi trois opérations distinctes, pouvant être organisées sans confondre leurs responsabilités.

5. Les fonctions propres au cœur : conception de l'éditeur

a. Une interface qui ne dépend pas entièrement des modules

Bien que les modules spécialisés assurent une partie importante des traitements de l'ECMMP, le cœur conserve ses propres fonctions.

Ces fonctions concernent principalement la structure générale de l'éditeur et les éléments graphiques qui ne relèvent pas directement d'un traitement linguistique ou d'un outil spécialisé.

Il peut notamment organiser les différentes zones de l'interface, définir leur position et gérer les éléments graphiques nécessaires à la présentation générale de l'environnement.

Cette responsabilité permet de conserver une cohérence visuelle entre les différents modules.

Le cœur ne délègue donc pas l'intégralité de l'affichage : il conserve la maîtrise de l'organisation générale de l'interface, tandis que les modules interviennent sur les éléments qui relèvent de leur spécialité.

b. Séparation entre interface générale et contenu linguistique

La conception de l'éditeur implique de distinguer deux niveaux d'affichage.

Le premier concerne l'interface générale : son organisation, ses zones de travail et ses éléments graphiques communs. Ces opérations relèvent du cœur.

Le second concerne la représentation du contenu du programme. Cette opération est destinée aux modules linguistiques, qui peuvent adapter l'affichage aux caractéristiques du langage sélectionné.

Cette répartition évite de mélanger la conception visuelle de l'IDE avec les règles de représentation propres à chaque langage.

Elle permet également d'envisager plusieurs modules linguistiques sans devoir redéfinir entièrement l'apparence générale de l'environnement pour chacun d'eux.

Le cœur conserve ainsi la structure commune de l'éditeur, tandis que les composants spécialisés peuvent intervenir à l'intérieur de cette structure.

c. Utilisation des fonctions graphiques de la NumWorks

L'interface de l'ECMMP est développée directement dans l'environnement graphique disponible sur la calculatrice.

Le cœur exploite notamment les fonctions de Kandinsky afin de construire et d'actualiser certains éléments de l'interface.

Les opérations graphiques doivent être organisées en tenant compte des dimensions de l'écran et des ressources disponibles.

La conception de l'éditeur doit donc privilégier une organisation adaptée à la taille de la calculatrice, en évitant de multiplier les opérations graphiques inutiles.

Cette contrainte matérielle influence directement la manière dont les fonctions propres au cœur sont conçues.

Elle impose de rechercher un équilibre entre la lisibilité de l'interface, la quantité d'informations affichées et le coût des traitements graphiques.

6. Limites et perspectives d'évolution du cœur

a. Gestion de la mémoire

La conservation du code source dans une chaîne de caractères unique simplifie l'organisation du contenu, mais elle présente certaines limites.

En Python, les chaînes de caractères sont immuables. Une opération de concaténation crée donc une nouvelle chaîne à partir des données précédentes et du nouveau contenu.

Lorsque le programme devient volumineux et que de nombreux caractères sont ajoutés successivement, ces opérations peuvent entraîner des allocations mémoire répétées.

Ce phénomène doit être pris en compte sur la NumWorks, dont la mémoire vive disponible pour MicroPython est limitée.

Le découpage temporaire du contenu en plusieurs chaînes pour les besoins de l'affichage entraîne également la création d'objets supplémentaires. Cette organisation facilite le rendu, mais son coût mémoire doit être évalué.

Une évolution possible consisterait à adopter une représentation du contenu mieux adaptée aux opérations d'édition, par exemple une structure organisée en lignes.

Il serait toutefois nécessaire de comparer les coûts de stockage et de manipulation des différentes approches avant de retenir une solution définitive.

b. Coût de la transmission des données

Les modules linguistiques doivent pouvoir traiter le contenu transmis par le cœur.

Lorsque le programme devient important, les opérations nécessaires à son analyse et à sa représentation peuvent devenir plus coûteuses.

Le projet envisage donc un découpage du contenu en tokens afin de structurer les informations avant leur traitement linguistique.

Cette approche pourrait faciliter le travail des modules en leur fournissant des unités mieux adaptées à l'analyse du programme.

Il faudra néanmoins veiller à ne pas créer une quantité excessive d'objets supplémentaires. La tokenisation peut elle-même nécessiter des allocations mémoire et son intérêt dépendra de la méthode de représentation retenue.

L'objectif ne consiste donc pas uniquement à réduire la taille des données transmises, mais à organiser leur traitement de manière plus efficace.

c. Organisation de la boucle principale

La boucle principale doit effectuer suffisamment de traitements pour maintenir une interface réactive, sans solliciter inutilement le processeur.

La répétition des appels aux composants constitue un point de vigilance. Certaines opérations peuvent être nécessaires à chaque itération, tandis que d'autres ne devraient intervenir qu'en présence d'un changement de données ou d'un événement particulier.

Une distinction plus précise entre les traitements permanents et les traitements ponctuels pourrait donc contribuer à améliorer les performances globales.

La gestion des appuis prolongés sur les touches constitue également une difficulté à prendre en compte. Une détection fondée uniquement sur l'état instantané d'une touche peut provoquer plusieurs traitements successifs lorsque celle-ci reste enfoncée.

Des mécanismes permettant de distinguer un nouvel appui du maintien d'une touche pourront être étudiés afin d'améliorer la précision des interactions.

d. Compatibilité et évolutivité des interfaces

La dépendance du cœur à un nombre limité de points d'entrée facilite l'intégration des modules, mais impose également de respecter certaines conventions.

Les paramètres transmis, les structures de données retournées et les responsabilités de chaque fonction doivent rester clairement définis.

Une modification non compatible de l'interface d'un module pourrait entraîner des erreurs lors de son utilisation par le cœur.

Il sera donc nécessaire de conserver une structure de communication stable, tout en permettant aux composants de faire évoluer leurs mécanismes internes.

Cette exigence est particulièrement importante pour les modules linguistiques, dont les besoins peuvent être différents selon les langages pris en charge.

e. Perspectives d'amélioration

Plusieurs améliorations pourront être envisagées pour renforcer le fonctionnement général du cœur.

La gestion du contenu pourra notamment être enrichie afin de prendre en charge les modifications à différentes positions, la suppression de caractères et l'organisation plus complète des lignes de code.

La gestion de l'affichage pourra également être optimisée pour éviter les opérations graphiques répétées lorsque le contenu ou l'interface ne nécessitent aucune actualisation.

Enfin, les échanges avec les modules pourront évoluer afin de prendre en charge des traitements plus structurés, notamment grâce à la préparation du contenu linguistique sous forme de tokens.

Ces évolutions devront conserver le principe fondamental de l'architecture : le cœur reste responsable de l'organisation générale et de la coordination, tandis que les composants spécialisés réalisent les traitements qui leur sont attribués.

Conclusion de la première couche

Le cœur de l'ECMMP constitue le point central de l'ensemble de l'architecture. Il organise les interactions entre les différents composants, conserve le contenu principal de l'éditeur et assure une partie de la construction de son interface.

Son fonctionnement repose sur une séparation des responsabilités : les opérations générales restent sous son contrôle, tandis que les traitements spécialisés sont confiés aux modules correspondants.

La limitation des points d'entrée à une fonction d'initialisation et une fonction principale d'exécution permet de conserver des échanges simples et cohérents. Le cœur peut ainsi solliciter les modules sans dépendre de leur fonctionnement interne.

Cette organisation doit permettre de faire évoluer progressivement l'environnement, tout en conservant une structure centrale relativement stable.

Le cœur constitue ainsi le socle sur lequel reposent les autres couches de l'ECMMP : il assure leur cohérence, leur coordination et leur intégration dans un environnement de développement unique.

B. Modules de langages et d'outils : deuxième couche

La deuxième couche de l'ECMMP regroupe les composants spécialisés qui assurent les fonctionnalités propres à l'environnement de développement.

Contrairement au cœur, dont la responsabilité principale est de coordonner le fonctionnement général de l'IDE, ces modules réalisent des traitements spécifiques. Leur organisation repose sur deux catégories distinctes : les modules de langages et les modules d'outils.

Les modules d'outils prennent en charge les interactions et les fonctionnalités nécessaires à l'utilisation de l'éditeur. Leur fonctionnement repose sur une organisation interne pouvant comporter de nombreuses fonctions auxiliaires, tout en conservant un nombre limité de points d'entrée accessibles au cœur.

Les modules de langages possèdent une architecture différente. Ils ont vocation à prendre en charge des traitements plus complexes, depuis l'analyse du contenu source jusqu'à sa représentation ou son exécution. Leur fonctionnement interne peut donc être particulièrement développé, même si leur communication avec le cœur repose sur une interface volontairement réduite.

Cette séparation permet de conserver une architecture générale commune, tout en adaptant le fonctionnement interne de chaque module à la nature des opérations qu'il doit effectuer.

1. Les modules de langages

Les modules de langages constituent une composante essentielle de l'ECMMP. Ils ont pour objectif de permettre à l'environnement de prendre en charge différents langages, chacun disposant de ses propres règles de représentation et, selon sa nature, de traitement.

Leur fonctionnement se distingue de celui des modules d'outils par la complexité des opérations qu'ils doivent réaliser.

Là où un outil peut généralement effectuer une opération ciblée à partir de paramètres transmis par le cœur, un module linguistique doit pouvoir traiter un ensemble de données représentant un programme complet.

Cette responsabilité implique plusieurs étapes : l'analyse du contenu, sa structuration sous une représentation intermédiaire, puis son exploitation pour produire un affichage ou effectuer une exécution.

a. Une architecture interne spécialisée

Une fonction principale, plusieurs traitements internes

L'organisation envisagée pour les modules linguistiques repose sur un principe différent de celui des modules d'outils.

Le cœur ne doit pas avoir à appeler individuellement toutes les fonctions nécessaires à l'analyse et au traitement d'un programme. Il lui suffit de solliciter le point d'entrée principal du module linguistique concerné.

À l'intérieur de celui-ci, plusieurs fonctions auxiliaires peuvent alors être utilisées successivement afin de réaliser l'ensemble des opérations nécessaires.

Le fonctionnement peut notamment être organisé autour de trois grandes étapes :

1. L'analyse du contenu transmis par le cœur ;
2. La construction d'une représentation structurée du programme ;
3. L'exploitation de cette représentation afin de réaliser son affichage ou son exécution.

Cette organisation permet de conserver une interface simple entre le cœur et le module, sans limiter la complexité de son fonctionnement interne.

Le nombre de fonctions accessibles depuis le cœur ne reflète donc pas nécessairement le nombre de fonctions présentes dans le module lui-même.

Une prise en charge propre à chaque langage

Chaque langage possède ses propres règles de syntaxe et de représentation. Un module destiné à HTML ne réalise donc pas exactement les mêmes opérations qu'un module destiné à Python.

Le premier doit notamment reconnaître les balises et organiser les éléments d'un document, tandis que le second doit être capable d'identifier des instructions et de traiter leur enchaînement.

L'architecture des modules linguistiques doit par conséquent permettre à chacun d'implémenter ses propres mécanismes d'analyse et de traitement.

L'objectif n'est pas d'imposer un fonctionnement interne identique à tous les langages, mais de leur fournir un mode de communication commun avec le cœur de l'ECMMP.

Cette distinction permet d'envisager l'intégration de langages aux caractéristiques très différentes, tout en conservant une architecture générale cohérente.

b. Analyse du contenu et identification des éléments du langage

Réception du code source

Lorsqu'un module linguistique est sollicité, le cœur lui transmet le contenu du programme sous la forme d'une chaîne de caractères.

Cette chaîne correspond au contenu principal conservé en mémoire par l'éditeur.

Le module doit alors analyser les données reçues afin d'identifier les éléments qui composent le programme et de déterminer leur signification en fonction des règles du langage concerné.

Cette étape constitue le point de départ du traitement linguistique.

Elle permet de passer d'un contenu textuel brut à des informations structurées, qui pourront ensuite être exploitées par les fonctions internes du module.

Identification lexicale et syntaxique

L'analyse d'un programme nécessite de distinguer plusieurs catégories d'éléments.

Dans un langage comme Python, il peut notamment s'agir des mots-clés, des identificateurs, des opérateurs, des constantes et des différentes structures syntaxiques.

Dans un langage de balisage comme HTML, l'analyse devra plutôt identifier les balises, leurs attributs, leur contenu et leurs relations structurelles.

Ces opérations peuvent être réparties entre plusieurs fonctions internes.

Une première étape peut effectuer une analyse lexicale, c'est-à-dire reconnaître les éléments élémentaires du contenu. Une seconde étape peut ensuite effectuer une analyse syntaxique, afin de déterminer comment ces éléments s'organisent entre eux.

L'objectif est de ne plus manipuler uniquement une succession de caractères, mais de disposer d'informations permettant de comprendre la structure du programme.

Cette distinction est particulièrement importante pour la suite du traitement : l'affichage et l'exécution ne doivent pas dépendre uniquement de la position d'un caractère dans la chaîne originale, mais également de sa fonction dans le langage.

Construction d'une représentation intermédiaire

À l'issue de cette analyse, le module doit pouvoir produire une structure de données plus facile à exploiter que le texte source initial.

Le projet envisage notamment l'utilisation d'une liste d'éléments appelés « intentions ».

Cette liste constitue une représentation intermédiaire du programme : elle rassemble les informations identifiées pendant l'analyse et les organise de manière à faciliter les opérations suivantes.

Chaque élément peut correspondre à une instruction, à une déclaration, à un élément de structure ou à une opération que le module devra traiter.

L'intérêt de cette méthode est de séparer l'analyse du texte de son exploitation.

La fonction chargée de l'exécution ou du rendu n'a ainsi plus besoin de parcourir directement l'ensemble du texte source pour tenter d'en déterminer la signification. Elle peut s'appuyer sur une structure déjà analysée et organisée.

Cette représentation intermédiaire pourra également permettre de regrouper plusieurs informations relatives à un même élément : son type, son contenu, ses paramètres ou encore les relations qu'il entretient avec d'autres éléments du programme.

Dans le cas d'un langage exécutable comme Python, cette organisation pourra se rapprocher d'une représentation intermédiaire d'instructions. Pour un langage de balisage comme HTML, une structure organisée en éléments et en relations hiérarchiques pourra être davantage adaptée.

Le terme « liste d'intentions » désigne ici le format de travail envisagé dans le cadre du projet, dont la structure définitive restera à préciser lors de l'implémentation.

Intérêt de cette étape de structuration

La création d'une représentation intermédiaire présente plusieurs avantages.

Premièrement, elle permet de séparer les responsabilités du module. L'analyse linguistique peut être réalisée indépendamment de l'exécution ou de l'affichage.

Deuxièmement, elle évite de devoir recommencer systématiquement l'analyse du texte lorsqu'une opération ultérieure nécessite de consulter les informations du programme.

Enfin, elle permet d'organiser le traitement du contenu sous une forme adaptée aux ressources et aux possibilités de MicroPython.

Cette étape constitue donc un élément central de l'architecture envisagée pour les modules linguistiques.

c. Exploitation de la représentation intermédiaire

Une fonction d'exécution ou de rendu

Une fois la représentation intermédiaire construite, une autre partie du module prend en charge son exploitation.

Cette fonction parcourt les éléments préalablement analysés et réalise les opérations associées à chacun d'eux.

La nature du traitement dépend du langage concerné.

Pour un langage destiné à produire une représentation graphique, comme HTML, le module devra transformer les informations structurées en éléments affichables.

Pour un langage exécutable, comme Python, il devra interpréter les instructions et appliquer les opérations correspondantes.

Cette séparation permet de distinguer la reconnaissance du langage de son exploitation.

La première étape détermine ce que contient le programme. La seconde exploite ces informations pour produire le résultat attendu.

Une représentation adaptée au rendu graphique

Dans le cas d'un module chargé de produire un affichage, les éléments identifiés doivent être transformés en opérations graphiques.

Le module linguistique peut ainsi parcourir sa représentation intermédiaire et déterminer quels éléments doivent être affichés, dans quel ordre et selon quelles caractéristiques.

Pour HTML, cela peut notamment concerner la disposition des éléments, leur contenu textuel et leurs propriétés de représentation.

L'objectif est de transformer une description textuelle en une représentation graphique exploitable par l'environnement.

Il ne s'agit donc pas nécessairement d'exécuter des instructions au sens d'un langage de programmation classique, mais de produire une représentation correspondant à la structure décrite par le document.

Une interprétation progressive des instructions

Pour les langages exécutables, le traitement consiste à parcourir les instructions et à réaliser les opérations qu'elles représentent.

Le module doit notamment être capable de conserver les informations nécessaires au déroulement du programme : les variables, les valeurs manipulées et la position de l'instruction en cours de traitement.

L'exécution peut être réalisée de manière progressive, en traitant successivement les éléments de la représentation intermédiaire.

Cette approche présente un intérêt particulier pour l'ECMMP : elle permet d'éviter de confier l'intégralité du traitement à une seule opération difficile à interrompre ou à contrôler.

Elle ouvre également la possibilité d'intégrer des mécanismes de pause, de reprise et de gestion des interactions utilisateur.

d. Gestion des résultats et communication avec l'interface

Retour des informations au cœur

Le module linguistique ne doit pas uniquement réaliser le traitement du programme. Il doit également transmettre au cœur les informations nécessaires à la poursuite du fonctionnement de l'ECMMP.

Ces informations peuvent notamment concerner :

- L'état du traitement ;
- Les éléments destinés à être affichés dans la console ;
- Les éventuelles erreurs détectées ;
- Les demandes d'interaction nécessitant une intervention de l'utilisateur ;
- Les informations permettant de poursuivre une exécution interrompue.

Le retour de la fonction doit donc permettre au cœur de connaître la situation du module sans avoir à examiner directement son fonctionnement interne.

Cette organisation conserve le principe général de communication entre les couches : le module réalise son traitement et transmet un résultat structuré au composant central.

Séparation entre résultat du programme et console

Il est important de distinguer le résultat produit par le programme du mécanisme chargé de l'afficher dans l'interface.

Le module linguistique doit déterminer les informations à transmettre, tandis que leur présentation dans la console appartient à l'organisation générale de l'ECMMP.

Par exemple, lorsqu'un programme produit une sortie textuelle, le module peut transmettre cette sortie au cœur sous la forme d'une chaîne ou d'une collection de messages.

Le cœur peut alors exploiter ces données pour les présenter dans la zone de console prévue à cet effet.

Cette séparation permet de conserver une interface commune, même lorsque les sorties produites par différents langages sont de nature différente.

Elle facilite également la gestion des erreurs et des messages d'information.

Gestion des erreurs

L'analyse et l'exécution d'un programme peuvent rencontrer différentes difficultés.

Une erreur peut notamment être détectée pendant l'analyse lexicale, lors de la vérification de la structure syntaxique ou au cours de l'exécution d'une instruction.

Le module linguistique doit donc être capable d'identifier les problèmes relevant de sa responsabilité et de transmettre les informations correspondantes au cœur.

Ces informations peuvent notamment préciser la nature de l'erreur et, lorsque cela est possible, l'emplacement concerné dans le programme.

Le cœur peut ensuite organiser leur affichage dans la console ou dans une zone réservée aux messages.

Cette séparation évite que les mécanismes de gestion des erreurs soient entièrement intégrés au programme principal.

e. Gestion des interactions pendant l'exécution

La problématique des instructions nécessitant une saisie

Certains langages permettent aux programmes de demander une information à l'utilisateur pendant leur exécution.

En Python, la fonction input() constitue un exemple représentatif de ce fonctionnement.

Lorsqu'un programme rencontre une telle instruction, son exécution ne peut pas simplement continuer comme si une valeur avait déjà été fournie.

Le module doit attendre que l'utilisateur communique l'information demandée, puis reprendre le traitement avec la valeur obtenue.

Cette situation introduit une difficulté particulière dans l'architecture de l'ECMMP : le module linguistique doit pouvoir demander une intervention sans bloquer définitivement le fonctionnement général de l'éditeur.

En effet, l'interface doit rester capable de recevoir les interactions de l'utilisateur pendant que le programme attend une saisie.

Il devient donc nécessaire de distinguer l'exécution du programme de la réception des données fournies par l'utilisateur.

Une exécution suspendue plutôt qu'un redémarrage

Une première approche consisterait à arrêter le traitement lorsqu'une saisie est nécessaire, puis à relancer le programme après réception de la réponse.

Cette méthode présenterait toutefois un problème majeur : le programme pourrait recommencer depuis son début, alors que certaines instructions auraient déjà été exécutées.

Pour éviter ce comportement, le projet envisage une gestion persistante de l'état d'exécution.

Cette méthode consiste à conserver les informations nécessaires pour reprendre le programme exactement à l'endroit où il a été interrompu.

Plusieurs données doivent alors être préservées :

- La position de l'instruction actuellement traitée ;
- Les variables et leurs valeurs ;
- Les informations internes nécessaires à la poursuite du programme ;
- La nature de l'interaction en attente ;
- Le contexte permettant de reprendre le traitement.

Ces informations constituent le contexte d'exécution du programme.

Lorsqu'une instruction nécessite une saisie, le module peut suspendre son traitement et transmettre au cœur un état indiquant qu'une intervention utilisateur est attendue.

L'ECMMP peut alors présenter la demande de saisie sans devoir recommencer l'exécution du programme.

Une fois la réponse obtenue, les données nécessaires sont transmises au module, qui peut reprendre le traitement à partir du contexte précédemment conservé.

Rôle du cœur dans la réception des entrées

Le cœur conserve son rôle de coordination pendant cette phase.

Le module linguistique indique qu'une saisie est nécessaire et fournit les informations permettant de présenter la demande à l'utilisateur.

Le cœur organise ensuite la réception de la réponse à travers les mécanismes d'interaction de l'ECMMP.

Une fois la saisie terminée, il transmet la valeur obtenue au module linguistique, accompagnée du contexte d'exécution nécessaire à la reprise.

Cette organisation permet d'éviter que chaque module linguistique ait besoin de gérer directement les périphériques d'entrée de la calculatrice.

Le module exprime un besoin fonctionnel ; le cœur se charge de coordonner la réponse correspondante.

Une fonction d'écoute ou un mécanisme de reprise

Une possibilité envisagée consiste à ajouter une fonction spécialisée dans la réception des réponses attendues par le module linguistique.

Cette fonction pourrait être chargée de vérifier si une interaction est en attente et de transmettre les informations nécessaires lorsque l'utilisateur a fourni une réponse.

Cependant, il n'est pas indispensable de créer un troisième point d'entrée public pour chaque module linguistique.

Une autre solution consiste à conserver un unique point d'entrée principal, appelé à différentes étapes du fonctionnement du programme, auquel le cœur transmet les événements ou les nouvelles données disponibles.

Le module peut alors déterminer son état courant et agir en conséquence :

- Lorsque le programme n'a pas encore commencé, il initialise son traitement ;
- Lorsque le programme est en cours d'exécution, il poursuit l'interprétation ;
- Lorsqu'une saisie est attendue, il conserve cet état et attend les données correspondantes ;

- Lorsque la réponse est reçue, il reprend l'exécution à partir de son contexte sauvegardé ;
- Lorsque le programme est terminé, il retourne son résultat final.

Cette approche permet de conserver un nombre réduit de points d'entrée tout en disposant d'un mécanisme de reprise suffisamment évolutif.

Elle constitue une piste particulièrement intéressante pour maintenir la réactivité de l'interface pendant l'exécution des programmes.

f. Limites et perspectives des modules linguistiques

Complexité de l'analyse des langages

La prise en charge de différents langages implique des niveaux de complexité variables.

Un langage simple à représenter peut nécessiter relativement peu de traitements, tandis qu'un langage de programmation complet implique de reconnaître de nombreuses structures syntaxiques et de gérer des comportements plus complexes.

La construction d'une représentation intermédiaire adaptée représente donc un travail important.

Il sera nécessaire de déterminer précisément les éléments que chaque module doit reconnaître et les informations qu'il doit conserver.

Coût mémoire de la représentation intermédiaire

La construction d'une liste d'éléments structurés implique également des allocations mémoire.

Plus un programme contient d'instructions ou d'éléments de structure, plus la représentation intermédiaire peut devenir volumineuse.

Cette contrainte est particulièrement importante sur la NumWorks.

Il faudra donc trouver un équilibre entre la richesse des informations conservées et la quantité de mémoire nécessaire à leur stockage.

Une représentation trop détaillée pourrait devenir coûteuse, tandis qu'une représentation trop simplifiée pourrait compliquer les opérations de traitement ultérieures.

Coût de l'exécution progressive

L'exécution d'un programme par interprétation de sa représentation intermédiaire implique de parcourir les éléments successivement.

Cette méthode permet un meilleur contrôle du déroulement, mais elle nécessite également de gérer correctement les variables, les structures de contrôle et les interactions.

L'exécution progressive devra donc être optimisée afin de ne pas ralentir excessivement l'utilisation de l'ECMMP.

La quantité d'instructions traitées à chaque étape pourra notamment être étudiée afin de préserver un équilibre entre la rapidité d'exécution et la disponibilité de l'interface.

Gestion des interruptions et reprise du contexte

La gestion des demandes de saisie représente une difficulté supplémentaire.

Le module doit pouvoir conserver suffisamment d'informations pour reprendre le traitement sans perdre les opérations déjà effectuées.

Cette nécessité implique de concevoir une représentation explicite de l'état d'exécution.

Il faudra également définir les comportements à adopter lorsqu'une exécution est interrompue volontairement, lorsqu'une erreur survient ou lorsque l'utilisateur souhaite abandonner le programme.

Ces mécanismes devront être développés progressivement et validés sur la calculatrice.

Perspectives d'évolution

L'architecture envisagée permet d'ouvrir plusieurs perspectives.

Les modules linguistiques pourront progressivement prendre en charge davantage de structures et de fonctionnalités propres à chaque langage.

La représentation intermédiaire pourra également être enrichie afin de faciliter l'analyse, l'exécution et le rendu.

Enfin, le mécanisme de gestion du contexte d'exécution pourra permettre d'intégrer des interactions plus avancées, comme les demandes de saisie, les interruptions ou la reprise d'un programme.

Cette organisation doit permettre de faire évoluer les modules linguistiques sans modifier profondément le fonctionnement général du cœur.

2. Les modules d'outils

Les modules d'outils constituent la seconde catégorie de composants de la deuxième couche.

Leur rôle est de fournir les fonctionnalités nécessaires à l'utilisation de l'ECMMP et de prendre en charge les interactions qui ne relèvent pas directement du traitement des langages.

Contrairement aux modules linguistiques, ils effectuent généralement des opérations plus ciblées. Leur fonctionnement interne peut néanmoins être composé de nombreuses fonctions auxiliaires, nécessaires à la préparation des données, à la réalisation des traitements et à leur actualisation.

Le cœur ne doit pas avoir à connaître tous ces mécanismes. Il utilise uniquement les points d'entrée prévus pour leur communication.

a. Organisation générale des modules d'outils

Deux points d'entrée principaux

L'architecture des modules d'outils repose sur deux fonctions principales accessibles au cœur.

La première est une fonction d'initialisation, chargée de préparer l'environnement propre au module.

La seconde est une fonction d'exécution, sollicitée pendant le fonctionnement général de l'IDE.

Ces deux fonctions constituent les points d'entrée principaux du composant.

Le module reste libre de contenir autant de fonctions auxiliaires que nécessaire pour effectuer ses opérations internes. Celles-ci ne sont cependant pas appelées directement par le cœur.

Cette organisation permet de limiter les dépendances et de conserver une interface de communication simple.

La fonction d'initialisation

La fonction d'initialisation intervient avant le fonctionnement courant du module.

Elle permet de préparer les structures de données, de définir les paramètres nécessaires et de retourner les informations qui devront être conservées pendant l'exécution de l'IDE.

Le cœur récupère ces informations et les transmettra de nouveau au module lorsque celui-ci sera sollicité.

Cette méthode permet de maintenir l'état du composant entre plusieurs appels.

Elle évite également d'avoir à reconstruire ses données internes à chaque itération de la boucle principale.

La fonction principale d'exécution

La seconde fonction est sollicitée pendant la boucle principale du cœur.

Elle reçoit les paramètres nécessaires au fonctionnement du module et réalise les opérations correspondant à son rôle.

Les informations transmises peuvent notamment concerner un événement détecté, une instruction à réaliser ou les données internes du composant.

Le module peut ensuite exploiter ses propres fonctions auxiliaires et retourner les informations actualisées au cœur.

La fonction principale constitue ainsi une interface entre le fonctionnement général de l'IDE et l'ensemble des traitements internes du module.

b. Le module de gestion du clavier : keyboard.py

Rôle du module

Le module keyboard.py est chargé de traduire les interactions physiques avec le clavier de la NumWorks en informations exploitables par l'éditeur.

Il constitue l'un des premiers outils nécessaires à l'utilisation de l'ECMMP, puisque la saisie du contenu du programme repose directement sur la capacité à reconnaître les touches pressées et les caractères qui leur correspondent.

Son fonctionnement repose notamment sur les constantes et les fonctions disponibles dans le module ion.

Organisation de la table de correspondance

La correspondance entre les touches physiques et les caractères est organisée dans une structure de données regroupant plusieurs informations.

Chaque entrée contient l'identifiant de la touche concernée ainsi que les caractères associés aux différents états du clavier.

Trois états sont actuellement prévus :

- L'état classique ;
- L'état Alpha ;
- L'état Shift.

Cette organisation permet de centraliser les associations entre les touches et les caractères au lieu de multiplier les conditions individuelles dans le programme.

La sélection du caractère à retourner dépend de l'état courant du clavier.

Gestion des états

Le module conserve une variable représentant l'état de saisie actuel.

Lorsqu'une touche de modification est détectée, l'état est actualisé afin de déterminer quelle association doit être utilisée lors de la prochaine saisie.

Le fonctionnement envisagé repose sur une distinction entre les modes classique, Alpha et Shift, permettant d'adapter les caractères produits sans modifier physiquement les touches de la calculatrice.

Cette gestion est réalisée indépendamment du contenu de l'éditeur : le module clavier retourne le caractère détecté, tandis que le cœur décide de la manière de l'exploiter.

Fonctionnement interne et retour au cœur

La fonction principale du module parcourt la table de correspondance afin de vérifier si une touche référencée est actuellement détectée.

Lorsqu'une correspondance est trouvée, le caractère associé à l'état courant est récupéré.

Le module retourne ensuite le caractère obtenu ainsi que l'état actualisé du clavier.

Le cœur peut alors intégrer le caractère au contenu de l'éditeur, sans avoir besoin de connaître la manière dont la correspondance a été déterminée.

Cette organisation permet de faire évoluer la table de correspondance et les mécanismes de détection sans modifier directement la logique de stockage du code source.

Perspectives d'amélioration

Plusieurs évolutions pourront être envisagées.

La détection des appuis prolongés devra notamment être affinée afin d'éviter certains changements d'état répétés lorsque les touches de modification restent maintenues.

La gestion de la suppression de caractères, de la répétition des touches et d'autres opérations de saisie pourra également être développée.

Ces améliorations devront conserver la séparation entre la détection des entrées physiques et leur exploitation par le cœur.

c. Le module de gestion du curseur : mouse.py

Rôle du module

Le module mouse.py est chargé de gérer un curseur graphique permettant de matérialiser une position d'interaction dans l'interface.

Il ne correspond pas à une souris physique, mais à un élément graphique déplacé à l'aide des touches directionnelles de la calculatrice.

Son objectif est de fournir un repère visuel mobile qui pourra accompagner les différentes opérations de l'éditeur.

Initialisation et conservation des données graphiques

L'affichage d'un curseur nécessite de conserver les informations relatives à la zone graphique située sous celui-ci.

En effet, lorsqu'il se déplace, il est nécessaire de restaurer l'arrière-plan de son ancienne position avant d'afficher sa nouvelle représentation.

Le module utilise notamment les possibilités offertes par Kandinsky pour récupérer et modifier les pixels concernés.

Lors de l'initialisation, les données graphiques nécessaires sont enregistrées afin de pouvoir être réutilisées lors des déplacements.

Cette préparation permet de ne pas perdre les éléments graphiques présents sous le curseur.

Déplacement et actualisation

Pendant le fonctionnement de l'ECMMP, le cœur transmet au module les informations nécessaires au déplacement du curseur.

Le module effectue alors les opérations graphiques correspondantes : restauration de la zone précédente, actualisation de la position et affichage du curseur à son nouvel emplacement.

Les informations actualisées sont ensuite retournées au cœur afin de conserver un état cohérent entre deux appels.

Cette méthode permet de déplacer le curseur sans avoir à reconstruire intégralement l'affichage de l'interface à chaque mouvement.

Intérêt de la séparation

L'ensemble des opérations graphiques relatives au curseur est regroupé dans le module mouse.py.

Le cœur n'a donc pas besoin de gérer directement la récupération et la restauration des pixels, ni de connaître l'ensemble des opérations nécessaires à chaque déplacement.

Il transmet les informations utiles, puis récupère les données actualisées.

Cette séparation facilite les évolutions du curseur et évite d'intégrer des traitements graphiques spécialisés dans la boucle principale.

d. Exploitation et coordination des modules d'outils

Une communication centralisée

Les modules d'outils sont intégrés au fonctionnement général par l'intermédiaire du cœur.

Celui-ci initialise leurs environnements, conserve les données retournées et les sollicite lorsque les opérations correspondantes sont nécessaires.

Cette organisation permet de centraliser la coordination sans imposer une connaissance détaillée du fonctionnement interne des modules.

Le cœur reste ainsi indépendant des fonctions auxiliaires propres à chaque outil.

Transmission ciblée des paramètres

Lorsqu'un module est sollicité, le cœur lui transmet uniquement les informations dont il a besoin pour effectuer son traitement.

Cette transmission ciblée limite les dépendances et évite de rendre accessibles inutilement toutes les données de l'environnement.

Elle facilite également la compréhension des interfaces : chaque module reçoit des paramètres correspondant directement à sa responsabilité.

Le retour de la fonction permet ensuite d'actualiser les informations conservées par le cœur.

Conservation de l'état des modules

Certains outils nécessitent de conserver des informations entre plusieurs appels.

C'est notamment le cas des données relatives à la position du curseur ou à l'état du clavier.

Le cœur conserve les structures retournées par les fonctions d'initialisation et les fonctions d'exécution, puis les réutilise lors des appels suivants.

Cette méthode permet de maintenir un état cohérent sans devoir réinitialiser continuellement les composants.

Elle constitue un élément important du fonctionnement des modules d'outils dans la boucle principale.

e. Limites et perspectives des modules d'outils

Coût des appels successifs

Les modules d'outils peuvent être sollicités de manière répétée pendant le fonctionnement de l'IDE.

Leur organisation doit donc limiter les opérations inutiles et éviter de reconstruire des données qui peuvent être conservées entre plusieurs appels.

Cette optimisation est particulièrement importante dans le contexte de la NumWorks, où les ressources matérielles disponibles sont limitées.

Gestion des états internes

La conservation des données entre plusieurs appels implique de maintenir la cohérence des informations retournées par les modules.

Une mauvaise gestion de ces structures pourrait entraîner des comportements inattendus dans l'interface.

Il est donc important de définir clairement le rôle de chaque donnée et de conserver des structures de retour cohérentes.

Évolution des outils

L'architecture actuelle permet d'envisager l'ajout de nouveaux modules d'outils.

Chaque nouveau composant devra toutefois respecter les conventions de communication prévues et présenter une responsabilité clairement définie.

Il devra également être évalué en fonction de son coût en ressources et de son utilité dans l'environnement.

L'objectif reste de conserver des modules indépendants, faciles à maintenir et suffisamment spécialisés pour remplir correctement leur fonction.

Conclusion de la deuxième couche

La deuxième couche constitue la partie spécialisée de l'architecture de l'ECMMP.

Elle repose sur deux catégories de modules dont les organisations internes répondent à des besoins différents.

Les modules de langages sont conçus comme des composants de traitement complexes. Ils doivent être capables d'analyser le contenu source, de produire une représentation intermédiaire, puis d'exploiter les informations obtenues afin de réaliser un rendu ou une exécution.

Les modules d'outils reposent sur une organisation plus ciblée. Ils possèdent leurs propres fonctions auxiliaires, mais leur communication avec le cœur est volontairement limitée à deux points d'entrée principaux : l'initialisation et l'exécution.

Cette distinction permet au cœur de conserver une interface générale de communication simple, tout en laissant aux composants spécialisés la liberté de mettre en œuvre les traitements nécessaires à leur fonctionnement.

L'ensemble constitue ainsi une architecture dans laquelle les responsabilités sont distribuées entre plusieurs composants indépendants, mais coordonnés par une structure centrale commune.

C. Compléments : troisième couche

La troisième couche de l'architecture de l'ECMMP correspond aux possibilités d'extension des modules existants. Contrairement aux deux premières couches, dont les rôles et les mécanismes de fonctionnement sont définis par la conception actuelle du projet, cette couche reste essentiellement prospective.

Son objectif est de ne pas limiter définitivement l'ECMMP aux fonctionnalités imaginées lors de son développement initial. En effet, un environnement modulaire peut présenter des possibilités d'évolution qui n'ont pas nécessairement été anticipées par son concepteur.

Un utilisateur ou un développeur extérieur pourrait ainsi identifier un besoin particulier et souhaiter enrichir un module existant, lui associer un mécanisme supplémentaire ou proposer une organisation différente de certaines fonctionnalités.

La troisième couche constitue donc une ouverture volontaire de l'architecture. Elle laisse la possibilité d'introduire des compléments sans considérer que l'organisation initiale de l'ECMMP doit rester immuable.

Il est toutefois important de préciser que cette couche n'a pas fait l'objet d'une implémentation complète dans le cadre du développement actuel. Les mécanismes présentés ci-dessous correspondent à des hypothèses d'évolution, dont la faisabilité et l'intérêt devront être évalués en fonction des besoins réels.

1. Principe général des compléments

a. Une architecture ouverte à l'évolution

Les deux premières couches permettent déjà de distinguer le cœur de l'IDE et les modules spécialisés. Cette organisation constitue une base permettant d'ajouter progressivement des fonctionnalités.

Cependant, la modularité ne garantit pas automatiquement qu'un module puisse être étendu sans aucune modification de son environnement.

Dans l'architecture actuelle, le cœur connaît les points d'entrée des modules qu'il utilise. Lorsqu'un nouveau composant doit intervenir dans le fonctionnement d'un outil existant, cette organisation peut nécessiter une adaptation des différents éléments concernés.

Un complément peut donc dépasser le simple ajout d'un fichier indépendant. Il peut impliquer une modification du cœur, une évolution du module d'origine et l'intégration d'un nouveau composant spécialisé.

Cette possibilité constitue le principe fondamental de la troisième couche : permettre d'enrichir l'architecture existante lorsque les besoins dépassent les fonctionnalités initialement prévues.

b. Des besoins qui n'ont pas nécessairement été anticipés

En tant que développeur initial de l'ECMMP, il est difficile d'identifier à l'avance l'ensemble des usages possibles de l'environnement.

Un module conçu pour répondre à un besoin précis peut, par exemple, être utilisé dans un contexte différent de celui envisagé lors de sa création.

Un autre développeur pourrait alors considérer que son fonctionnement mérite d'être complété, adapté ou associé à de nouvelles fonctionnalités.

Cette situation pourrait notamment se produire lorsqu'un utilisateur souhaite modifier profondément le comportement d'un outil existant, sans vouloir remplacer entièrement le composant d'origine.

Il devient alors intéressant de disposer d'une architecture dans laquelle les modules peuvent eux-mêmes être enrichis par d'autres composants.

La troisième couche ne cherche donc pas à répondre à un besoin fonctionnel unique. Elle vise plutôt à conserver une possibilité d'évolution pour des besoins qui pourraient apparaître au fur et à mesure de l'utilisation de l'ECMMP.

2. Compléments des modules d'outils

a. Modulation de modules pour des modules

L'une des évolutions envisageables consiste à permettre à un module supplémentaire d'intervenir directement dans le fonctionnement d'un module d'outil existant.

Cette organisation introduirait une relation de dépendance supplémentaire entre les composants.

Prenons l'exemple du module keyboard.py. Celui-ci possède déjà son propre fonctionnement interne et communique avec le cœur par l'intermédiaire de ses points d'entrée principaux.

Un développeur extérieur pourrait cependant identifier un intérêt à lui ajouter un mécanisme particulier, nécessitant une organisation différente de la gestion des entrées clavier.

Dans une telle situation, le simple ajout d'une fonction indépendante ne serait pas nécessairement suffisant.

Il pourrait être nécessaire de modifier plusieurs éléments :

- **Le cœur de l'IDE,** afin d'adapter les appels et la transmission des informations au nouveau fonctionnement ;
- **Le module d'origine,** afin de prendre en compte les modifications nécessaires à son interface ou à son comportement ;
- **Le nouveau module complémentaire,** chargé de réaliser les traitements supplémentaires.

Le complément viendrait ainsi enrichir un composant déjà intégré à l'architecture, plutôt que de fonctionner comme un outil totalement indépendant.

Cette organisation pourrait conduire à introduire un niveau supplémentaire de dépendance, susceptible d'être considéré comme une quatrième couche dans certains cas.

Il ne s'agit toutefois pas d'une nouvelle couche systématique : un simple ajout de fonctionnalité ne suffit pas, à lui seul, à justifier une nouvelle couche architecturale. Celle-ci dépendrait surtout du rôle du composant et de sa position dans les relations entre modules.

L'exemple permet néanmoins d'illustrer une possibilité intéressante : un module pourrait devenir lui-même le support d'autres modules, au lieu de constituer uniquement un composant directement sollicité par le cœur.

b. Adaptation de l'architecture et préservation de sa cohérence

L'ajout d'un complément peut entraîner des modifications qui dépassent son propre fonctionnement interne.

En effet, si le nouveau composant modifie la manière dont un outil reçoit ou transmet ses informations, le cœur doit continuer à pouvoir communiquer correctement avec l'ensemble.

Il peut donc être nécessaire de reprendre les appels concernés dans le fichier principal, de modifier les structures de données utilisées par le module d'origine et de définir précisément les échanges avec le complément.

Cette adaptation est importante pour éviter que les différents composants ne fonctionnent selon des conventions incompatibles.

Une intégration incorrecte pourrait notamment entraîner des incohérences dans les paramètres transmis, des appels redondants ou des conflits entre les responsabilités des modules.

Le développement de compléments impose donc de conserver une vision globale de l'architecture, même lorsque la modification semble initialement limitée à un seul outil.

Il faudra également tenir compte des dépendances introduites. Un complément ne doit pas rendre le fonctionnement général de l'IDE excessivement dépendant d'un composant particulier, au risque de compliquer les évolutions ultérieures.

La troisième couche doit donc permettre une extension raisonnée de l'architecture, et non une accumulation de modules dont les relations deviendraient difficiles à maintenir.

3. Compléments des modules linguistiques

a. Extension et création de modules originaux

Les modules linguistiques constituent également des composants susceptibles de bénéficier de compléments.

Leur architecture interne étant particulièrement développée, il est envisageable qu'un développeur souhaite ajouter des traitements supplémentaires à un module existant.

Ces compléments pourraient notamment concerner l'analyse du contenu, la représentation des éléments du langage ou certaines opérations propres à son traitement.

Un module linguistique pourrait ainsi être enrichi sans que l'ensemble de son fonctionnement initial soit nécessairement remplacé.

Une autre possibilité consisterait à développer des composants spécialisés destinés à être utilisés par plusieurs modules linguistiques, lorsque ceux-ci présentent des besoins communs.

Cette approche pourrait permettre de mutualiser certains mécanismes et d'éviter de reproduire plusieurs fois des traitements similaires.

Il s'agit toutefois de possibilités théoriques. Aucun mécanisme universel de complément linguistique n'est actuellement défini dans le projet.

L'organisation exacte dépendrait donc des besoins rencontrés et de la nature des modifications souhaitées.

b. Limites de conception et d'ajout de modules

La création de compléments supplémentaires soulève plusieurs difficultés.

La première concerne la compatibilité avec les modules existants. Un complément doit pouvoir communiquer avec le composant auquel il est destiné, sans compromettre le fonctionnement attendu par le cœur.

La deuxième concerne la complexité des dépendances. Plus les composants s'appuient les uns sur les autres, plus il devient nécessaire de documenter précisément leurs interfaces et leurs responsabilités.

La troisième concerne les ressources matérielles. Chaque composant supplémentaire peut entraîner une augmentation de la mémoire utilisée et du nombre de traitements effectués. Cette contrainte devra rester particulièrement importante dans un environnement MicroPython fonctionnant sur la NumWorks.

Enfin, l'ajout de compléments peut rendre la maintenance plus complexe, notamment si plusieurs développeurs font évoluer indépendamment des composants qui dépendent les uns des autres.

Il sera donc nécessaire de définir des conventions communes pour faciliter leur intégration.

Ces conventions pourraient notamment préciser les paramètres attendus, les structures de données retournées et les conditions d'utilisation des composants complémentaires.

4. Limites actuelles et perspectives

a. Une couche volontairement prospective

Contrairement aux deux premières couches, la troisième ne répond pas à un besoin immédiat identifié lors du développement de l'ECMMP.

En tant que concepteur du projet, je n'ai pas nécessairement connaissance de toutes les utilisations que d'autres développeurs pourraient faire de l'environnement. Il serait donc prématuré de définir une liste exhaustive de compléments ou d'imposer un fonctionnement précis à cette couche.

Son maintien dans l'architecture répond à une volonté différente : ne pas fermer artificiellement les possibilités d'évolution du projet.

L'absence de besoin immédiat ne signifie pas que cette couche est inutile. Elle indique simplement que son intérêt dépendra des usages futurs de l'ECMMP et des besoins qui pourront émerger après sa diffusion.

Il est ainsi possible qu'un développeur identifie une possibilité d'extension à laquelle le concepteur initial n'avait pas pensé.

Cette liberté constitue précisément l'intérêt recherché.

b. Le rôle de futurs développeurs

L'ECMMP étant conçu comme un environnement modulaire, son évolution pourrait ne pas dépendre uniquement de son développeur initial.

D'autres utilisateurs pourraient souhaiter adapter certains composants, proposer de nouvelles fonctionnalités ou développer des modules répondant à des besoins particuliers.

La troisième couche laisse donc volontairement une place à ces contributions.

Un développeur pourrait notamment choisir de compléter un outil existant, de créer un composant intermédiaire ou de modifier les relations entre plusieurs modules afin de proposer un fonctionnement différent.

Ces évolutions pourraient conduire à des architectures plus complexes que celle initialement prévue.

Il faudra toutefois veiller à ce que ces modifications restent compatibles avec les ressources disponibles et avec les conventions de communication de l'ECMMP.

c. Une évolution conditionnée par les besoins réels

L'architecture ne doit pas être complexifiée uniquement pour anticiper des besoins hypothétiques.

Chaque complément devra présenter un intérêt identifiable et justifier les modifications qu'il implique.

Il sera notamment nécessaire d'évaluer :

- La fonctionnalité supplémentaire réellement apportée ;
- Les modifications nécessaires dans les composants existants ;
- Les nouvelles dépendances introduites ;
- Le coût supplémentaire en mémoire et en calcul ;
- La facilité de maintenance de l'ensemble.

Cette démarche permettra de distinguer les extensions réellement utiles des ajouts qui complexifieraient l'environnement sans apporter de bénéfice suffisant.

L'objectif n'est donc pas de rechercher systématiquement une architecture à quatre couches ou davantage, mais de laisser la possibilité de l'atteindre lorsque les besoins le justifient.

Conclusion de la troisième couche

La troisième couche représente une ouverture volontaire de l'architecture de l'ECMMP vers des possibilités d'extension qui ne sont pas nécessairement connues au moment de sa conception.

Elle permet d'envisager l'ajout de composants complémentaires, susceptibles d'enrichir les modules existants ou de modifier certaines relations entre les différentes parties de l'environnement.

Cette couche ne constitue pas une fonctionnalité actuellement implémentée, mais une perspective laissée aux futurs développeurs et utilisateurs du projet.

Son intérêt repose sur une idée simple : un environnement modulaire doit pouvoir évoluer au-delà des besoins initialement identifiés par son concepteur.

L'ECMMP ne cherche donc pas à imposer une architecture définitivement figée. Il propose une première organisation cohérente, tout en laissant volontairement la possibilité de la compléter lorsque de nouveaux usages le justifieront.

Rapport de veille technologique

Sommaire de la veille

1. Article 1 — ViperIDE : programmer depuis un navigateur 53
2. Article 2 — Pico VS Code : une extension pour simplifier le développement 55
3. Article 3 — Visual Studio Code sur Raspberry Pi 57
4. Synthèse générale 59

Article 1 — ViperIDE : programmer depuis un navigateur

Titre : *How to write code for your Raspberry Pi Pico in your web browser with ViperIDE*
Auteur : Les Pounder
Source : Tom's Hardware
Date : 13 juillet 2024

https://www.tomshardware.com/raspberry-pi/raspberry-pi-pico/how-to-write-code-for-your-raspberry-pi-pico-in-your-web-browser-with-viperide

Présentation de l'article

L'article de Tom's Hardware présente ViperIDE, un environnement de développement créé par Volodymyr Shymanskyy et accessible directement depuis un navigateur web. Il permet de programmer des cartes électroniques avec MicroPython sans avoir à installer un éditeur de code complet sur son appareil. Comme il fonctionne dans un navigateur, il peut être utilisé depuis différents supports : ordinateur, Chromebook ou même smartphone.

ViperIDE rassemble plusieurs outils dans une seule interface : un éditeur de code, un terminal interactif (REPL), un gestionnaire de fichiers et un gestionnaire de paquets. Ce dernier permet notamment d'ajouter des bibliothèques sur la carte utilisée. L'article montre cette possibilité avec une bande de LED NeoPixel, dont le fonctionnement nécessite l'installation d'un module supplémentaire.

L'outil propose également plusieurs moyens de communiquer avec la carte électronique, notamment par USB, Bluetooth ou WebREPL. Il est ainsi possible d'écrire, d'envoyer et d'exécuter du code depuis le navigateur.

Analyse des avantages et des limites

Le principal avantage de ViperIDE est son accessibilité. L'utilisateur n'a pas besoin d'installer un environnement de développement complet et peut retrouver ses outils sur différents appareils. La présence d'un gestionnaire de paquets est également intéressante, puisqu'elle permet d'ajouter des bibliothèques selon les besoins du projet, sans devoir les intégrer toutes dès le départ.

Cependant, cette solution nécessite un appareil externe pour utiliser le navigateur et communiquer avec la carte électronique. L'environnement de développement et le matériel sur

lequel le programme s'exécute restent donc séparés. Son fonctionnement dépend également de la possibilité d'établir une connexion entre les deux appareils.

Comparaison avec l'ECMMP

ECMMP adopte une approche différente. Son principal avantage est de réunir l'environnement de développement et le matériel d'exécution sur un seul appareil : la calculatrice NumWorks. Il n'est donc pas nécessaire d'utiliser un ordinateur ou un smartphone pour écrire et exécuter les programmes.

En revanche, cette autonomie impose certaines limites. Contrairement à ViperIDE, ECMMP ne dispose pas d'un gestionnaire de paquets permettant d'ajouter librement des bibliothèques. Il doit fonctionner avec les modules disponibles dans la version de MicroPython intégrée à Epsilon. Il n'offre pas non plus le même accès aux possibilités matérielles de la carte, puisque le développement s'effectue à travers les fonctions mises à disposition par le système de la calculatrice.

Ces deux solutions répondent donc à des besoins différents : ViperIDE privilégie l'accessibilité et la possibilité d'ajouter des fonctionnalités, tandis qu'ECMMP cherche à proposer un environnement de développement directement intégré à son appareil d'exécution.

Article 2 — Pico VS Code : une extension pour simplifier le développement

Titre : *Boost Your Pico Projects with the new Pico VS Code Extension*
Auteur : Paul Oberosler
Source : Raspberry Pi
Date : 18 septembre 2024

https://www.raspberrypi.com/news/pico-vscode-extension/

Présentation de l'article

L'article publié sur le site officiel de Raspberry Pi présente Pico VS Code, une extension destinée à Visual Studio Code et conçue pour faciliter le développement sur les cartes Raspberry Pi Pico. Au moment de la publication, l'extension est présentée en version bêta publique.

Son objectif principal est de simplifier la configuration des projets, qui peut être assez compliquée dans le domaine du développement embarqué. En effet, les développeurs doivent parfois installer et configurer plusieurs outils, comme des kits de développement (SDK), des systèmes de compilation et des chaînes d'outils.

L'extension propose notamment un assistant de création de projets, des exemples déjà préparés et des outils pour gérer leur configuration. Elle prend en charge le développement en C/C++ ainsi que la création de projets en MicroPython. Dans ce dernier cas, elle s'appuie sur l'extension MicroPico pour envoyer et gérer les programmes sur la carte.

Pour les projets en C/C++, l'extension permet également de compiler le programme, de transférer le résultat sur la carte et d'utiliser des outils de débogage. Elle intègre enfin une documentation accessible directement dans Visual Studio Code, même sans connexion Internet.

Analyse des avantages et des limites

Le principal avantage de Pico VS Code est de rendre le développement sur Raspberry Pi Pico plus accessible. Les assistants et les configurations automatiques évitent à l'utilisateur de devoir tout préparer manuellement. Il peut ainsi se concentrer davantage sur son programme que sur l'installation des outils nécessaires.

L'intégration de la documentation est également intéressante : les informations utiles peuvent être consultées directement depuis l'éditeur, sans devoir ouvrir plusieurs pages sur Internet. Les outils de débogage apportent aussi un réel confort pour les projets en C/C++.

En revanche, cette solution nécessite un ordinateur sur lequel Visual Studio Code et ses extensions peuvent être installés. Certaines fonctionnalités avancées, comme le débogage matériel, demandent également du matériel supplémentaire, tel que le Raspberry Pi Debug Probe. L'environnement est donc plus complet, mais sa mise en place reste liée à une machine externe et à ses différents outils.

Comparaison avec ECMMP

ECMMP fait le choix inverse en intégrant directement l'édition et l'exécution des programmes dans la calculatrice NumWorks. Il n'est donc pas nécessaire de disposer d'un ordinateur pour développer et tester un script dans son environnement habituel.

Cependant, cette autonomie se paie par des possibilités plus limitées. Là où Pico VS Code propose des outils avancés de compilation, de débogage et de documentation, ECMMP doit fonctionner avec les moyens disponibles dans Epsilon. Son interface est également adaptée aux dimensions de l'écran et aux ressources de la calculatrice.

Cette comparaison montre deux approches différentes du développement embarqué. Pico VS Code cherche à faciliter le travail du développeur grâce à un environnement de bureau complet, tandis qu'ECMMP cherche à réunir les outils essentiels directement sur la machine utilisée pour exécuter les programmes.

Article 3 — Visual Studio Code sur Raspberry Pi

Titre : *Visual Studio Code comes to Raspberry Pi*
Auteur affiché : Ashley (l'article donne également la parole à Jim Bennett, de Microsoft)
Source : Raspberry Pi
Date : 12 février 2021

https://www.raspberrypi.com/news/visual-studio-code-comes-to-raspberry-pi/

Présentation de l'article

L'article publié sur le site officiel de Raspberry Pi présente l'arrivée de Visual Studio Code sur les ordinateurs Raspberry Pi. Jusqu'alors principalement utilisé sur Windows, MacOS et Linux pour processeurs x64, l'éditeur devient également disponible avec des versions compatibles ARM et ARM64, permettant son installation sur Raspberry Pi OS.

Visual Studio Code est un éditeur de code qui peut être complété par de nombreuses extensions. Il permet notamment de travailler avec différents langages comme Python, JavaScript, TypeScript ou C/C++. Il propose également des outils de gestion de versions avec Git et peut être enrichi pour programmer des microcontrôleurs grâce à des extensions comme PlatformIO.

L'article présente notamment l'exemple du Raspberry Pi 400, sur lequel Visual Studio Code peut être utilisé avec un écran, un clavier et une souris. L'intérêt est de permettre aux utilisateurs, notamment aux débutants, de retrouver un environnement de développement complet sur un ordinateur compact et abordable. Ils peuvent ainsi utiliser le même outil pour apprendre plusieurs langages et réaliser différents types de projets.

Analyse des avantages et des limites

Le principal avantage de cette solution est sa polyvalence. Grâce aux extensions, Visual Studio Code peut être adapté à de nombreux langages et usages. L'utilisateur peut donc conserver un seul éditeur pour différents projets, sans devoir apprendre à utiliser plusieurs logiciels.

L'installation de cet environnement sur un Raspberry Pi permet également de profiter d'outils de développement complets sur un ordinateur de petite taille et relativement accessible.

Cependant, Visual Studio Code reste un logiciel de bureau qui fonctionne sur un système d'exploitation complet. Il faut donc disposer d'un ordinateur capable de faire fonctionner Raspberry Pi OS et l'éditeur. L'utilisation présentée dans l'article nécessite également des périphériques habituels, comme un écran et un clavier. Cette solution reste donc différente d'un environnement directement intégré à un appareil spécialisé.

Comparaison avec ECMMP

ECMMP reprend une partie de cette idée : proposer plusieurs outils de développement au sein d'un même environnement, plutôt que de multiplier les logiciels indépendants. Toutefois, il pousse cette intégration plus loin en fonctionnant directement sur la calculatrice NumWorks, sans nécessiter d'ordinateur externe.

Cette approche permet d'avoir un environnement de développement portable et autonome, mais impose aussi des limites importantes. L'écran de la calculatrice est beaucoup plus petit que celui

d'un ordinateur, les possibilités de saisie sont différentes et les ressources disponibles sont bien plus réduites. ECMMP ne peut donc pas proposer le même confort d'utilisation ni le même choix de fonctionnalités que Visual Studio Code.

Ces deux projets montrent qu'un environnement de développement peut être adapté à des machines très différentes. Visual Studio Code cherche à rendre un ordinateur compact aussi polyvalent qu'un poste de travail classique, tandis qu'ECMMP cherche à intégrer directement les fonctions essentielles d'un éditeur de code dans une calculatrice.

Synthèse générale — Environnements de développement sur appareils compacts et systèmes embarqués

Les trois articles étudiés présentent différentes solutions permettant de développer des programmes sur des appareils compacts ou destinés à piloter du matériel électronique. Bien que leurs fonctionnements soient différents, ils cherchent tous à rendre le développement plus accessible, en proposant des outils adaptés aux besoins des utilisateurs.

Le premier article présente ViperIDE, un environnement accessible depuis un navigateur web. Son principal avantage est de pouvoir être utilisé sur différents appareils sans installer un logiciel complet. Il intègre également plusieurs outils utiles, comme un éditeur de code, un terminal interactif et un gestionnaire de paquets. Cette dernière fonctionnalité permet notamment d'ajouter des bibliothèques selon les besoins. Cependant, son utilisation nécessite un appareil externe et une connexion avec la carte électronique sur laquelle le programme doit être exécuté.

Le deuxième article présente Pico VS Code, une extension destinée à simplifier le développement sur les cartes Raspberry Pi Pico. Elle permet de retrouver plusieurs outils directement dans Visual Studio Code, notamment des assistants de configuration, de la documentation et des fonctions de compilation ou de débogage pour les projets concernés. Cette approche offre davantage de confort au développeur et facilite la mise en place des projets. En revanche, elle nécessite un ordinateur sur lequel installer et utiliser l'environnement de développement.

Enfin, le troisième article présente l'arrivée de Visual Studio Code sur Raspberry Pi OS. Contrairement aux deux solutions précédentes, il ne s'agit pas d'un outil spécifiquement consacré au développement MicroPython sur microcontrôleur, mais d'un véritable éditeur polyvalent installé directement sur un petit ordinateur. Grâce à ses nombreuses extensions, il permet de travailler sur différents langages et types de projets. Cette solution offre donc une grande liberté, mais nécessite un système d'exploitation complet et des ressources matérielles suffisantes pour faire fonctionner l'éditeur.

Bilan des avantages et des limites

Ces trois solutions montrent qu'il existe plusieurs manières d'adapter un environnement de développement à un appareil compact. Certaines privilégient l'accessibilité depuis un navigateur, d'autres enrichissent un éditeur existant grâce à des extensions, tandis que d'autres encore cherchent à faire fonctionner un environnement de bureau complet sur un ordinateur de petite taille.

Ces approches présentent plusieurs avantages communs : elles regroupent différents outils dans un même environnement, facilitent l'écriture et l'exécution des programmes et permettent d'ajouter des fonctionnalités selon les besoins. Elles offrent ainsi un confort de développement important et évitent parfois à l'utilisateur de devoir configurer lui-même l'ensemble des outils nécessaires.

Cependant, elles reposent généralement sur un appareil capable d'accueillir l'environnement de développement, en plus du matériel sur lequel les programmes doivent éventuellement être exécutés. Cette séparation peut nécessiter une connexion entre les deux appareils et impose de disposer d'un ordinateur ou d'un autre support externe. De plus, les environnements les plus

complets peuvent demander davantage de ressources matérielles et une configuration plus importante.

Mise en perspective avec ECMMP

Notre projet ECMMP adopte une approche différente de celles présentées dans les articles. Son objectif est de réunir directement l'édition et l'exécution des programmes dans un seul appareil : la calculatrice NumWorks. Contrairement aux solutions étudiées, il n'est donc pas nécessaire de disposer d'un ordinateur externe pour développer et tester des scripts dans l'environnement prévu.

Cette autonomie représente le principal avantage de notre projet. Elle permet de disposer d'un environnement de programmation directement sur la calculatrice, sans avoir à installer ou configurer un logiciel sur une autre machine. Elle s'accompagne également d'une volonté de modularité, afin de pouvoir organiser les différentes fonctionnalités d'ECMMP en plusieurs modules.

En contrepartie, le développement sur NumWorks impose des contraintes bien plus importantes. Les ressources matérielles sont limitées, l'écran réduit les possibilités de l'interface et la version de MicroPython intégrée à Epsilon ne propose pas toutes les fonctionnalités et bibliothèques que l'on pourrait retrouver dans d'autres environnements. Il est donc nécessaire d'adapter le code, de limiter certaines opérations et de chercher constamment à optimiser l'utilisation de la mémoire et de l'espace de stockage.

La comparaison de ces différentes solutions montre finalement qu'il n'existe pas une seule manière de concevoir un environnement de développement. Un éditeur complet offre davantage de fonctionnalités et de confort, tandis qu'un environnement embarqué peut privilégier l'autonomie et la simplicité d'utilisation. Chaque approche implique des choix et des compromis en fonction du matériel disponible et des objectifs recherchés.

ECMMP s'inscrit ainsi dans cette seconde approche : plutôt que de chercher à reproduire toutes les possibilités d'un éditeur de bureau comme Visual Studio Code, le projet cherche à en reprendre certains principes, notamment la modularité, tout en les adaptant aux capacités et aux limites de la calculatrice NumWorks.

Conclusion générale

A. Bilan du rapport technique

L'étude du support a permis de mieux comprendre les possibilités et les limites de la calculatrice NumWorks. Son microcontrôleur, sa mémoire et son environnement Epsilon offrent une base suffisante pour expérimenter un éditeur de code, mais les ressources disponibles restent limitées. Les essais réalisés sur l'affichage, les chaînes de caractères et la gestion de la mémoire montrent qu'il faut faire attention à la taille des données et au nombre d'opérations effectuées. Il faut aussi tenir compte des différences entre le MicroPython de la calculatrice et un Python installé sur un ordinateur.

À partir de ces contraintes, le rapport a présenté une architecture organisée en trois couches. Le cœur assure la coordination générale et conserve certaines fonctions propres à l'éditeur. Les modules de deuxième couche prennent en charge les traitements spécialisés : les modules de langages s'occupent de l'analyse, de la représentation ou de l'exécution du code, tandis que les modules d'outils gèrent notamment les interactions avec le clavier et le curseur. La troisième couche ouvre des possibilités d'extension, mais reste volontairement prospective.

Il faut toutefois distinguer l'architecture décrite de ce qui est déjà réalisé. Les premiers éléments du cœur et les modules keyboard.py et mouse.py servent à mettre en place et à tester cette organisation. En revanche, plusieurs fonctions présentées dans le rapport restent à développer ou à valider, en particulier la prise en charge complète des langages, leur représentation intermédiaire et les mécanismes d'extension. Le rapport décrit donc à la fois des réalisations, des choix de conception et des pistes de travail : il ne présente pas l'ECMMP comme un IDE déjà terminé.

La suite du développement devra surtout permettre de rendre l'édition du code réellement utilisable, de compléter les interactions, puis d'intégrer progressivement un premier module de langage. Chaque étape devra être testée directement sur la calculatrice afin de vérifier les performances, la consommation mémoire et la réactivité de l'interface. Les choix d'architecture pourront ensuite être ajustés à partir des résultats obtenus.

B. Apports de la veille et perspectives

La veille technologique a permis d'étudier trois approches différentes. ViperIDE propose un environnement accessible depuis un navigateur. Pico VS Code ajoute des outils spécialisés à un éditeur déjà connu. Enfin, Visual Studio Code sur Raspberry Pi montre qu'un éditeur complet peut fonctionner sur un ordinateur compact. Ces exemples ne répondent pas exactement au même besoin, mais ils montrent plusieurs façons de regrouper des outils et de faciliter le développement sur des appareils de petite taille.

La principale différence avec l'ECMMP est le support utilisé. Les solutions étudiées s'appuient sur un navigateur ou sur un ordinateur capable d'exécuter un environnement de développement. L'ECMMP cherche au contraire à fonctionner directement sur la NumWorks, qui sert à la fois de support de développement et de machine d'exécution pour les programmes concernés. Cette autonomie est intéressante, mais elle impose des compromis : écran réduit, saisie moins confortable, mémoire limitée et fonctionnalités MicroPython moins nombreuses.

Cette veille ne fournit donc pas un modèle à reproduire tel quel. Elle aide plutôt à identifier des principes utiles, comme la modularité, le regroupement des outils et l'adaptation de l'environnement au matériel disponible. Elle permet aussi de mieux situer le projet : son intérêt ne vient pas du fait qu'il reprendrait toutes les possibilités d'un éditeur comme Visual Studio Code, mais de la recherche d'une solution adaptée à une calculatrice et à ses contraintes.

L'ECMMP reste un projet en cours de développement. La priorité est maintenant de transformer progressivement l'architecture proposée en fonctionnalités utilisables, sans perdre de vue les limites du support. Les évolutions futures devront répondre à des besoins réels et apporter un avantage suffisant pour justifier leur coût en mémoire, en calcul et en maintenance. C'est ainsi que le projet pourra gagner en fonctionnalités tout en restant adapté à la NumWorks.